

# GRIP-VR: Grasp Recognition for Intention Prediction in Virtual Reality

*Emmanouil Karydis*



Thesis submitted in partial fulfillment of the requirements for the

*Masters' of Science degree in Computer Science*

University of Crete

School of Sciences and Engineering

Computer Science Department

Voutes University Campus, 700 13 Heraklion, Crete, Greece

Thesis Advisor: Prof. *George Papagiannakis*

Co-Supervisor: Dr. *Emmanouil Kamarianakis*

---

This work has been performed at the University of Crete, School of Sciences and Engineering, Computer Science Department.

The work has been supported by the Foundation for Research and Technology - Hellas (FORTH), Institute of Computer Science (ICS).



UNIVERSITY OF CRETE  
COMPUTER SCIENCE DEPARTMENT

**GRIP-VR: Grasp Recognition for Intention Prediction in Virtual Reality**

Thesis submitted by  
**Emmanouil Karydis**  
in partial fulfillment of the requirements for the  
Masters' of Science degree in Computer Science

THESIS APPROVAL

Author: \_\_\_\_\_  
Emmanouil Karydis

Committee approvals: \_\_\_\_\_  
George Papagiannakis  
Professor, Thesis Supervisor

\_\_\_\_\_  
Antonis Argyros  
Professor, Committee Member

\_\_\_\_\_  
Anastasios Roussos  
Principal Researcher, Committee Member

Departmental approval: \_\_\_\_\_  
Antonios Argyros  
Professor, Director of Graduate Studies

Heraklion, June 2026



# GRIP-VR: Grasp Recognition for Intention Prediction in Virtual Reality

## Abstract

Intuitive and physically plausible hand-object interaction is a critical bottleneck in first-person spatial computing. In egocentric Extended Reality (XR) environments, modern hardware utilizes outward-facing cameras and machine learning estimators to track the user’s skeletal hand joints in real-time. However, mapping continuous, high-dimensional joint-angle coordinates directly to rigid virtual objects frequently causes severe physics engine failures (such as mesh penetration and visual clipping) and latency spikes. These spikes violate the strict motion-to-photon latency budgets (typically 11.1 milliseconds in 90Hz systems) necessary to maintain presence and prevent cybersickness. Traditional robotic and computer vision approaches attempt to solve this either via continuous joint-angle trajectory planning, which is highly sensitive to sensor noise and self-occlusions, or through text-guided grasp generation, which requires explicit user input and breaks natural immersion.

To resolve these limitations, this thesis introduces GRIP-VR (Grasp Recognition for Intention Prediction in Virtual Reality), an end-to-end "Intent-to-Operation" framework. GRIP-VR shifts the interaction paradigm from continuous joint-angle tracking to discrete, semantic intention classification. The framework models intention using the 33 distinct grasp configurations of the Feix Grasp Taxonomy. To bypass the severe compute and VRAM constraints of standalone VR headsets, we design a decoupled client-server architecture: the VR headset serves as a lightweight rendering and tracking client, while a local high-performance server hosts a fine-tuned, parameter-efficient Large Language Model (Meta LLaMA 3.2 3B Instruct) that serves as the reasoning backend. The LLM predicts contextually valid grasp taxonomy subsets by fusing multi-modal inputs: 3D object shape embeddings generated via the OpenShape library, scene-level spatial obstructions (blocked axes), and physical object scale. A custom Multi-Layer Perceptron (MLP) projector aligns the 1280-dimensional OpenShape embeddings into four virtual tokens spliced directly into the LLM’s vocabulary latent space.

To protect the real-time runtime loop from network latency and generative inference delays, the Unity frontend executes a pre-computation ("baking") phase during scene initialization. The frontend queries the AI server sequentially across six orthogonal approach directions, caching a complete spatial map of valid interactions for every object. During active runtime, raw egocentric hand tracking captured via the Unity XR Hands API is stabilized using a velocity-damped forward prediction heuristic and a magnetic pinch occlusion solver. The stabilized joint metrics are classified into candidate Feix grasp types using a boundary-penalty

scoring heuristic with metric weights optimized via Differential Evolution to maximize cluster separability. Finally, a local C# Runtime JOIN set-operation algorithm dynamically filters the pre-baked grasp list against the active approach vector and environmental raycasts in under 1 millisecond.

Empirical evaluations demonstrate that the fine-tuned generative backend achieves a Mean Precision of 80.45% and a Mean Recall of 76.08% on a validation split of completely unseen object geometries, rising to 90.06% semantic accuracy at the functional megacluster level. The model maintains a 99.8% Valid Format Rate, ensuring seamless remote execution. The local C# constraint solver executes in approximately 0.50 milliseconds, effortlessly satisfying the rendering constraints of a Meta Quest 3 headset at 90Hz. Finally, zero-shot comparative experiments against the state-of-the-art DexTOG framework show that GRIP-VR provides dense semantic compression, enabling upstream Vision-Language Models to accurately reverse-engineer high-level human functional intent from discrete grasp taxonomy states.

# GRIP-VR: Αναγνώριση Λαβής για Πρόβλεψη Πρόθεσης σε Περιβάλλοντα Εικονικής Πραγματικότητας

## Περίληψη

Η διαισθητική και φυσικά ρεαλιστική αλληλεπίδραση χεριού-αντικειμένου αποτελεί ένα κρίσιμο σημείο συμφόρησης (bottleneck) στον χωρικό υπολογισμό πρώτου προσώπου (first-person spatial computing). Σε εγωκεντρικά περιβάλλοντα Εκτεταμένης Πραγματικότητας (XR), το σύγχρονο υλικό (hardware) χρησιμοποιεί κάμερες με κατεύθυνση προς τα έξω και εκτιμητές μηχανικής μάθησης για την παρακολούθηση των αρθρώσεων του σκελετού του χεριού του χρήστη σε πραγματικό χρόνο. Ωστόσο, η απευθείας αντιστοίχιση συνεχών, υψηλής διάστασης συντεταγμένων των γωνιών των αρθρώσεων σε στερεά εικονικά αντικείμενα προκαλεί συχνά σοβαρές αστοχίες της μηχανής φυσικής (physics engine), όπως η διείσδυση πλέγματος (mesh penetration) και η οπτική αποκοπή (visual clipping), καθώς και απότομες αυξήσεις στην καθυστέρηση (latency spikes). Αυτές οι εξάρσεις παραβιάζουν τα αυστηρά όρια καθυστέρησης από την κίνηση στο φωτόνιο (motion-to-photon latency budgets, συνήθως 11.1 χιλιοστά του δευτερολέπτου σε συστήματα 90Hz), τα οποία είναι απαραίτητα για τη διατήρηση της αίσθησης παρουσίας και την αποτροπή της κυβερνοναυτίας (cybersickness). Οι παραδοσιακές προσεγγίσεις της ρομποτικής και της υπολογιστικής όρασης επιχειρούν να επιλύσουν το πρόβλημα είτε μέσω του συνεχούς σχεδιασμού τροχιάς των γωνιών των αρθρώσεων, ο οποίος είναι ιδιαίτερα ευαίσθητος στον θόρυβο των αισθητήρων και στις επικαλύψεις (self-occlusions), είτε μέσω της παραγωγής λαβής καθοδηγούμενης από κείμενο, η οποία απαιτεί ρητή εισαγωγή από τον χρήστη και διακόπτει τη φυσική εμπύθιση.

Για την επίλυση αυτών των περιορισμών, η παρούσα διατριβή παρουσιάζει το GRIP-VR (Grasp Recognition for Intention Prediction in Virtual Reality), ένα ολοκληρωμένο (end-to-end) πλαίσιο ‘Πρόθεσης-σε-Λειτουργία’ (Intent-to-Operation). Το GRIP-VR μετατοπίζει το παράδειγμα αλληλεπίδρασης από τη συνεχή παρακολούθηση των γωνιών των αρθρώσεων στη διακριτή, σημασιολογική ταξινόμηση της πρόθεσης. Το πλαίσιο μοντελοποιεί την πρόθεση χρησιμοποιώντας τους 33 διακριτούς σχηματισμούς λαβής της Ταξινόμιας Λαβών του Feix (Feix Grasp Taxonomy). Για την παράκαμψη των αυστηρών περιορισμών υπολογιστικής ισχύος και VRAM των αυτόνομων (standalone) συσκευών VR, σχεδιάσαμε μια αποσυνδεδεμένη αρχιτεκτονική πελάτη-διακομιστή (client-server): το σύστημα VR λειτουργεί ως ένας ελαφρύς πελάτης απόδοσης και παρακολούθησης (rendering and tracking client), ενώ ένας τοπικός διακομιστής υψηλής απόδοσης φιλοξενεί ένα μικρο-συντονισμένο (fine-tuned), αποδοτικό ως προς τις παραμέτρους Μεγάλο Γλωσσικό Μοντέλο (Meta LLaMA 3.2 3B Instruct) που λειτουργεί ως το σύστημα συλλογιστικής (reasoning backend). Το LLM προβλέπει τα σημασιολογικά έγκυρα υποσύνολα της ταξινόμιας λαβών συγχωνεύοντας πολυτροπικά δεδομένα εισόδου: ενσωματώσεις σχήματος τρισδιάστατων αντικειμένων (3D object shape embeddings) που παράγονται μέσω της βιβλιοθήκης

OpenShape, χωρικά εμπόδια σε επίπεδο σκηνής (μπλοκαρισμένοι άξονες) και τη φυσική κλίμακα του αντικειμένου. Ένας προσαρμοσμένος προβολέας Πολυστρωματικού Perceptron (MLP) ευθυγραμμίζει τις ενσωματώσεις ΟπενΣηαπε των 1280 διαστάσεων σε τέσσερα εικονικά διακριτικά (virtual tokens) που εισάγονται απευθείας στον λανθάνοντα χώρο του λεξιλογίου του LLM.

Για την προστασία του βρόχου εκτέλεσης (runtime loop) πραγματικού χρόνου από την καθυστέρηση του δικτύου και τις καθυστερήσεις του παραγωγικού συμπερασμού, το περιβάλλον διεπαφής (frontend) του Unity εκτελεί μια φάση προ-υπολογισμού (“baking”) κατά την αρχικοποίηση της σκηνής. Το frontend υποβάλλει ερωτήματα στον διακομιστή τεχνητής νοημοσύνης διαδοχικά σε έξι ορθογώνιες κατευθύνσεις προσέγγισης, αποθηκεύοντας στην κρυφή μνήμη έναν πλήρη χωρικό χάρτη έγκυρων αλληλεπιδράσεων για κάθε αντικείμενο. Κατά την ενεργή εκτέλεση, η ακατέργαστη εγωκεντρική παρακολούθηση χεριού (raw egocentric hand tracking) που καταγράφεται μέσω του Unity XR Hands API σταθεροποιείται χρησιμοποιώντας έναν ευρετικό αλγόριθμο πρόβλεψης προς τα εμπρός με απόσβεση ταχύτητας (velocity-damped forward prediction heuristic) και έναν επιλύτη απόκρυψης μαγνητικούτσιμπήματος (magnetic pinch occlusion solver). Οι σταθεροποιημένες μετρικές των αρθρώσεων ταξινομούνται σε υποψήφιους τύπους λαβής Feix χρησιμοποιώντας έναν ευρετικό αλγόριθμο βαθμολόγησης με ποινή ορίων (boundary-penalty scoring heuristic), με βάση μετρικών βελτιστοποιημένα μέσω Διαφορικής Εξέλιξης για τη μεγιστοποίηση της διαχωρισιμότητας των συστάδων (cluster separability). Τέλος, ένας τοπικός αλγόριθμος πράξεων συνόλων C\* Runtime JOIN φιλτράρει δυναμικά την προ-υπολογισμένη λίστα λαβών σε σχέση με το ενεργό διάνυσμα προσέγγισης και τις ανιχνεύσεις ακτίνων του περιβάλλοντος (environmental raycasts) σε λιγότερο από 1 χιλιοστό του δευτερολέπτου.

Εμπειρικές αξιολογήσεις καταδεικνύουν ότι το μικρο-συντονισμένο σύστημα παραγωγής πετυχαίνει Μέση Ακρίβεια (Mean Precision) 80.45% και Μέση Ανάκληση (Mean Recall) 76.08% σε ένα σύνολο επικύρωσης (validation split) εντελώς άγνωστων γεωμετριών αντικειμένων, ανεβαίνοντας στο 90.06% σε σημασιολογική ακρίβεια στο επίπεδο των λειτουργικών υπερ-συστάδων (functional megaclusters). Το μοντέλο διατηρεί ένα 99.8% Ποσοστό Έγκυρης Μορφοποίησης (Valid Format Rate), εξασφαλίζοντας απρόσκοπτη απομακρυσμένη εκτέλεση. Ο τοπικός επιλύτης περιορισμών σε C\* εκτελείται σε περίπου 0.50 χιλιοστά του δευτερολέπτου, ικανοποιώντας αβίαστα τους περιορισμούς απόδοσης ενός ακουστικού Meta Quest 3 στα 90Hz. Τέλος, συγκριτικά πειράματα μηδενικής εκμάθησης (zero-shot) έναντι του προηγμένου (state-of-the-art) πλαισίου DexTOG δείχνουν ότι το GRIP-VR παρέχει πυκνή σημασιολογική συμπίεση, επιτρέποντας σε ανώτερα (upstream) Μοντέλα Όρασης-Γλώσσας να ανασυνθέσουν (reverse-engineer) με ακρίβεια την υψηλού επιπέδου ανθρώπινη λειτουργική πρόθεση από διακριτές καταστάσεις της ταξινόμιας λαβών.

# Declaration of Authorship

I, Emmanouil Karydis, declare that this thesis titled, “GRIP-VR: Grasp Recognition for Intention Prediction in Virtual Reality”, and the work presented in it are my own. I confirm that:

- This work was done wholly or mainly while in candidature for a research degree at this University.
- Where any part of this thesis has previously been submitted for a degree or any other qualification at this University or any other institution, this has been clearly stated.
- Where I have consulted the published work of others, this is always clearly attributed.
- Where I have quoted from the work of others, the source is always given. With the exception of such quotations, this thesis is entirely my own work.
- I have acknowledged all main sources of help.
- Where the thesis is based on work done by myself jointly with others, I have made clear exactly what was done by others and what I have contributed myself.

**Signature:** Emmanouil Karydis

---

**Date:** June 13, 2026



# Acknowledgements

I would like to express my deepest gratitude, first and foremost, to my professors and advisors, Prof. George Papagiannakis and Dr. Manos Kamarianakis. I am also deeply indebted to Prof. Antonis Argyros for his expert advice and guidance on hand tracking and computer vision. Their invaluable guidance, continuous cooperation, and academic insight were instrumental in the successful completion of this thesis.

I am profoundly grateful to my family for their unwavering daily support, patience, and encouragement throughout the entirety of my studies. Their presence has been a constant source of strength in both my academic pursuits and everyday life.

Special thanks are owed to my close friends—George, Vasilis, Antonis, Anthi, Thomas, Sifis, Manos, Antreas, and Sotiris. Your camaraderie, moral support, and ability to keep me grounded during the most challenging and demanding periods of this journey mean the world to me.

Finally, I would extend my sincere appreciation to my fellow students and colleagues, Despoina, Manos, and Vasia, as well as to Alexandra and George, for their shared wisdom, collaborative spirit, and constant support.



# Contents

|          |                                                               |           |
|----------|---------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                           | <b>1</b>  |
| 1.1      | Area Overview and Motivation . . . . .                        | 1         |
| 1.2      | Research Objectives . . . . .                                 | 2         |
| 1.3      | Proposed Approach . . . . .                                   | 4         |
| 1.4      | Contributions of this Work . . . . .                          | 5         |
| 1.5      | Thesis Structure . . . . .                                    | 6         |
| <b>2</b> | <b>Background and Related Work</b>                            | <b>7</b>  |
| 2.1      | Robotic Grasping and Taxonomy . . . . .                       | 7         |
| 2.1.1    | Foundational Grasp Taxonomies . . . . .                       | 8         |
| 2.1.2    | Simulation and Large-Scale Grasp Synthesis . . . . .          | 9         |
| 2.1.3    | Bridging Human Priors and Robotic Kinematics . . . . .        | 10        |
| 2.2      | Generative AI and Multimodal Learning . . . . .               | 11        |
| 2.2.1    | Large Language Models as Planners and Policy Generators       | 11        |
| 2.2.2    | 3D Object Representation and Affordance Grounding . . .       | 13        |
| 2.2.3    | Semantic Intention Prediction and Task-Oriented Grasping      | 14        |
| 2.3      | Spatial Computing and Hand Tracking . . . . .                 | 15        |
| 2.3.1    | XR Hand Tracking Capabilities and Limitations . . . . .       | 15        |
| 2.3.2    | Semantic Affordances and Multi-Object Context . . . . .       | 16        |
| 2.3.3    | Pose versus Intent and VR Latency Constraints . . . . .       | 16        |
| <b>3</b> | <b>Dataset Engineering and Processing</b>                     | <b>18</b> |
| 3.1      | Physics Simulation and Evaluation Setup . . . . .             | 18        |
| 3.1.1    | Optimization of the Evaluation Pipeline . . . . .             | 19        |
| 3.2      | Data Extraction Pipeline . . . . .                            | 20        |
| 3.3      | Dataset Curation and Formatting for LLM Fine-Tuning . . . . . | 21        |
| 3.3.1    | Mathematical Representation of Constraint Blocking . . . .    | 22        |
| 3.3.2    | Ethical and Privacy Considerations . . . . .                  | 23        |
| <b>4</b> | <b>Multimodal LLM Architecture</b>                            | <b>24</b> |
| 4.1      | Model Selection and Baseline Architecture . . . . .           | 24        |
| 4.2      | Input Modalities and Embedding Strategy . . . . .             | 25        |
| 4.3      | Fine-Tuning Methodology . . . . .                             | 27        |

|          |                                                                         |           |
|----------|-------------------------------------------------------------------------|-----------|
| <b>5</b> | <b>XR System Architecture and Implementation</b>                        | <b>29</b> |
| 5.1      | System Overview and Client-Server Communication . . . . .               | 29        |
| 5.1.1    | The Inference Server and 4-Bit Quantization . . . . .                   | 30        |
| 5.1.2    | The Communication Pipeline . . . . .                                    | 30        |
| 5.2      | The Pre-Computation (“Baking”) Phase . . . . .                          | 30        |
| 5.2.1    | Initializing Object Scripts and Mesh Extraction . . . . .               | 31        |
| 5.2.2    | The 6-Axis Sequential Web Request Pipeline . . . . .                    | 31        |
| 5.2.3    | Caching the Validated Grasps . . . . .                                  | 32        |
| 5.3      | Dynamic Constraint Handling . . . . .                                   | 32        |
| 5.3.1    | Defining Axis Constraints and Approach Vectors . . . . .                | 33        |
| 5.4      | Real-Time Hand Intention Classification . . . . .                       | 34        |
| 5.4.1    | Predictive XR Kinematics Capture . . . . .                              | 34        |
| 5.4.2    | Affordance-Aligned Kinematic Evaluation . . . . .                       | 36        |
| 5.4.3    | Weight Optimization via Differential Evolution . . . . .                | 36        |
| <b>6</b> | <b>Experiments and Evaluation</b>                                       | <b>40</b> |
| 6.1      | Model Performance Metrics . . . . .                                     | 40        |
| 6.1.1    | Quantitative Accuracy Metrics . . . . .                                 | 40        |
| 6.1.2    | Semantic and Cluster-Level Accuracy . . . . .                           | 41        |
| 6.2      | Kinematic Classifier Quantitative Validation . . . . .                  | 42        |
| 6.2.1    | Differential Evolution Optimization on Synthetic Ground Truth . . . . . | 42        |
| 6.2.2    | Grasp Classification Confusion Matrix Analysis . . . . .                | 43        |
| 6.2.3    | PCA Visual Cluster Separability Analysis . . . . .                      | 44        |
| 6.3      | System and Performance Profiling . . . . .                              | 45        |
| 6.3.1    | Compute and Hardware Overhead . . . . .                                 | 46        |
| 6.3.2    | Frame Rate Stability during Runtime . . . . .                           | 47        |
| 6.4      | System Capabilities and Generalization . . . . .                        | 47        |
| 6.4.1    | Real-Time Environmental Awareness and Adaptation . . . . .              | 47        |
| 6.4.2    | Bimanual Support and Object Generalization . . . . .                    | 47        |
| 6.5      | From Taxonomic Grasp to Semantic Intent . . . . .                       | 48        |
| 6.5.1    | Contextual Limitations of SOTA Systems . . . . .                        | 48        |
| 6.5.2    | Evaluation Experiment using DexTOG . . . . .                            | 49        |
| 6.5.3    | VLM Intent Inference via Semantic Compression . . . . .                 | 49        |
| 6.5.4    | Conclusion of Findings . . . . .                                        | 50        |
| <b>7</b> | <b>Conclusion and Future Work</b>                                       | <b>52</b> |
| 7.1      | Summary of Achievements . . . . .                                       | 52        |
| 7.1.1    | Resolution of Research Questions . . . . .                              | 52        |
| 7.2      | System Limitations . . . . .                                            | 53        |
| 7.3      | Future Directions . . . . .                                             | 55        |
| 7.3.1    | On-Device AI and the Removal of Pre-Computation . . . . .               | 55        |
| 7.3.2    | Upstream Integration with Generative Grasp Synthesis . . . . .          | 55        |
| 7.3.3    | Autonomous Semantic Agents . . . . .                                    | 56        |

|       |                                                                                               |    |
|-------|-----------------------------------------------------------------------------------------------|----|
| 7.3.4 | Multi-Agent and Collaborative Interactions . . . . .                                          | 56 |
| 7.3.5 | Direct Multi-Modal Vision-Language Models for Tracking-<br>to-Intent Reconstruction . . . . . | 56 |

## **Appendices . . . . . 61**

### **A Supplementary Project Details . . . . . 63**

|       |                                                |    |
|-------|------------------------------------------------|----|
| A.1   | Code Availability . . . . .                    | 63 |
| A.2   | System Requirements and Installation . . . . . | 63 |
| A.2.1 | DexGraspBench and Dataset Setup . . . . .      | 63 |
| A.2.2 | GRIP-VR and Server Setup . . . . .             | 64 |
| A.2.3 | Running the Inference Server . . . . .         | 64 |
| A.2.4 | Unity Project Setup and Execution . . . . .    | 64 |
| A.3   | API Endpoints for the Python Server . . . . .  | 65 |
| A.4   | Example JSON Payloads . . . . .                | 65 |
| A.4.1 | Example Request Payload . . . . .              | 65 |
| A.4.2 | Example Response Payload (Success) . . . . .   | 66 |
| A.4.3 | Example Response Payload (Error) . . . . .     | 66 |



# List of Tables

|     |                                                                                                                          |    |
|-----|--------------------------------------------------------------------------------------------------------------------------|----|
| 4.1 | Complete training hyperparameters for the GRIP-VR multimodal LLM. . . . .                                                | 28 |
| 5.1 | Classification mapping of the 33 Feix Taxonomy grasp types into eight functional megaclusters. . . . .                   | 37 |
| 6.1 | Baseline quantitative prediction accuracy metrics of the fine-tuned LLaMA model on the validation set. . . . .           | 41 |
| 6.2 | Comparison of prediction accuracy between Strict Taxonomy (Class-Level) and Semantic (Megacluster-Level) levels. . . . . | 42 |
| 6.3 | Runtime and initialization system latency breakdown across the pipeline phases. . . . .                                  | 45 |



# List of Figures

|     |                                                                                                                                        |    |
|-----|----------------------------------------------------------------------------------------------------------------------------------------|----|
| 1.1 | The Grasp Type Taxonomy proposed by Thomas Feix, outlining the 33 human grasp configurations (reproduced from [1]). . . . .            | 3  |
| 1.2 | Conceptual architecture and system execution sequence of the GRIP-VR framework. . . . .                                                | 4  |
| 2.1 | The thumb position criteria (adduction vs. abduction) used in the Feix Grasp Taxonomy (reproduced from [1]). . . . .                   | 9  |
| 2.2 | Visual examples of multi-fingered grasps generated in the BODex dataset (reproduced from [2]). . . . .                                 | 9  |
| 2.3 | Visual examples of taxonomically complete hand poses from the Dexonomy dataset (reproduced from [3]). . . . .                          | 10 |
| 2.4 | Structural overview of the Flamingo visual language model (reproduced from [4]). . . . .                                               | 12 |
| 2.5 | The Robotics Transformer (RT-1) architecture mapping vision and language inputs to tokenized control primitives (reproduced from [5]). | 12 |
| 2.6 | Representation of multimodal joint embedding space in the OpenShape framework (reproduced from [6]). . . . .                           | 13 |
| 2.7 | Semantic grasping interaction interface within a virtual reality environment (reproduced from [7]). . . . .                            | 14 |
| 2.8 | Optimization of MANO hand models aligned with physical object contact heatmaps in ContactPose (reproduced from [8]). . . . .           | 15 |
| 2.9 | The Affordpose dataset construction process (reproduced from [9]).                                                                     | 16 |
| 3.1 | Physics-based multi-fingered grasp simulation executed inside the MuJoCo engine (reproduced from [10]). . . . .                        | 19 |
| 3.2 | A sample from the dataset created for fine tuning a small LLM for GRIP-VR. . . . .                                                     | 22 |
| 4.1 | The LLaMA 3.2 3B Instruct transformer block architecture (reproduced from [11]). . . . .                                               | 25 |
| 4.2 | Schematic representation of OpenShape 3D embeddings projected into the 3072-dimensional space of LLaMA (reproduced from [6]).          | 26 |
| 4.3 | The custom training evaluation callback captured metrics (Precision, Recall, F1-score). . . . .                                        | 28 |

|     |                                                                                                                                                                               |    |
|-----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 5.1 | Serialized list structure displaying baked valid grasp configurations cached locally inside Unity. . . . .                                                                    | 32 |
| 5.2 | Skeletal tracking capture and grasp predictions in the Unity XR frontend. . . . .                                                                                             | 39 |
| 6.1 | The classification confusion matrix heatmap of the 33 Feix grasp types based on optimized kinematic weights. . . . .                                                          | 43 |
| 6.2 | The Unity Grasp Landscape (Weighted PCA) showing the spatial distribution and separability of the 8 functional megaclusters. . . .                                            | 44 |
| 6.3 | Task Manager performance metrics showing 7.1 GB of 8.0 GB dedicated VRAM used (88.75%) and 100% GPU compute utilization during initialization. . . . .                        | 46 |
| 6.4 | Dynamic scene constraints adaptation inside the virtual scene. . .                                                                                                            | 47 |
| 6.5 | Symmetric bimanual grasp configuration alignment. . . . .                                                                                                                     | 48 |
| 6.6 | Multiple task-oriented dexterous hand poses generated through the DexTOG framework (reproduced from [12]). . . . .                                                            | 49 |
| 6.7 | Visual evaluation and intent inference verification during the spray bottle experiment. . . . .                                                                               | 50 |
| 6.8 | The multimodal Vision-Language Model interface demonstrating successful high-level intent reconstruction from discrete taxonomic grasp inputs (reproduced from [13]). . . . . | 51 |

# List of Abbreviations

| Abbreviation | Expansion                         |
|--------------|-----------------------------------|
| <b>API</b>   | Application Programming Interface |
| <b>CMC</b>   | Carpometacarpal                   |
| <b>DE</b>    | Differential Evolution            |
| <b>DFC</b>   | Distance to Force Closure         |
| <b>DGN</b>   | DexGraspNet                       |
| <b>DoF</b>   | Degrees of Freedom                |
| <b>GELU</b>  | Gaussian Error Linear Unit        |
| <b>GFM</b>   | GitHub Flavored Markdown          |
| <b>GPU</b>   | Graphics Processing Unit          |
| <b>HTTP</b>  | Hypertext Transfer Protocol       |
| <b>JSON</b>  | JavaScript Object Notation        |
| <b>LLM</b>   | Large Language Model              |
| <b>LoRA</b>  | Low-Rank Adaptation               |
| <b>MHA</b>   | Multi-Head Attention              |
| <b>MLP</b>   | Multi-Layer Perceptron            |
| <b>OOM</b>   | Out-Of-Memory                     |
| <b>PEFT</b>  | Parameter-Efficient Fine-Tuning   |
| <b>RQ</b>    | Research Question                 |
| <b>SOTA</b>  | State Of The Art                  |
| <b>TOG</b>   | Task-Oriented Grasping            |
| <b>VLA</b>   | Vision-Language-Action            |
| <b>VLM</b>   | Vision-Language Model             |
| <b>VR</b>    | Virtual Reality                   |
| <b>VRAM</b>  | Video Random Access Memory        |
| <b>WSL</b>   | Windows Subsystem for Linux       |
| <b>XR</b>    | Extended Reality                  |



# Chapter 1

## Introduction

### 1.1 Area Overview and Motivation

Intuitive and physically plausible hand-object interaction remains a fundamental challenge in spatial computing and virtual reality (VR), echoing long-standing complexities in robotic grasp choice and modeling. At the core of this challenge is the complex problem of hand pose prediction. While modern Extended Reality (XR) hardware provides real-time optical hand tracking, translating raw kinematic data into stable virtual grasps is notoriously difficult. Optical tracking is inherently prone to occlusion, noise, and jitter. When this noisy data is continuously mapped directly to a virtual hand attempting to grasp a rigid object, the result is often visual penetration, physically impossible joint angles, or a complete failure of the simulated physics.

Many existing systems attempt to solve hand pose prediction by relying on explicit user input, such as textual commands like “hold the cup” [14, 15]. While these text-to-grasp approaches have proven highly effective in the field of robotics for planning stable manipulation, they fundamentally contradict the design philosophy of VR. In immersive spatial computing environments, user interactions must be seamless, implicit, and natural. Forcing a user to rely on explicit textual or verbal prompts to trigger an interaction breaks the immersion that VR is uniquely designed to provide.

Alternatively, systems often utilize the user’s real-time hand data as it approaches an object to predict and dynamically generate the final continuous grasp pose [16]. However, beyond the inherent tracking inaccuracies in VR, this approach struggles significantly with geometric and mechanical complexity. Predicting highly intricate interactions—such as correctly threading fingers through the loops of scissors or establishing the precise, multi-finger pinch required for chopsticks—is exceptionally difficult to compute dynamically. Furthermore, while it is possible to reasonably predict the general spatial positioning of the hand as it approaches an object, the final joint kinematics are strictly dependent on the specific morphology of the virtual hand model itself. A continuous pose generated based

on the user’s physical hand tracking may not map successfully to a different VR avatar hand or a simulated robotic end-effector.

This highlights a critical bottleneck in current interaction pipelines: attempting to solve the continuous, high-degree-of-freedom problem of exact joint kinematics in real-time is both computationally expensive and highly fragile, often exceeding the strict motion-to-photon latency budgets required to maintain immersion and prevent cybersickness in VR [17]. To achieve stable, immersive interactions, there is a critical need to approach intention prediction through a more general and abstract lens. By dividing this complex geometric problem into simpler, discrete stages—specifically, by predicting abstract grasp types based on early approach dynamics rather than continuous joint angles [18]—systems can intelligently anticipate valid, physically grounded interactions.

## 1.2 Research Objectives

This thesis aims to develop a novel, multimodal framework for grasp type prediction and execution in Extended Reality (XR) environments. By shifting the paradigm from rigid, continuous pose-matching to discrete, intent-based grasp classification, this work seeks to synthesize generative artificial intelligence with spatial computing. The overarching goal is to create a system that understands both the geometric affordances of a virtual object and the morphological intent of the user’s hands, bridging the gap between human interaction and robotic grasping theory.

To achieve this, it is first necessary to translate the continuous, noisy data of optical hand tracking into a structured, semantic format. While taxonomies such as the Feix framework [1] provide a comprehensive categorization of robotic and human grasps, applying these abstract labels to real-time spatial data requires defining precise kinematic boundaries. The 33 distinct grasp types outlined by this taxonomy are shown in Figure 1.1.

Establishing clear parametric thresholds—such as specific finger joint angles, palm orientations, and spatial hand data areas—is crucial for distinguishing subtle variations in user intent.

Once these grasp types can be reliably categorized from hand metrics, the system must be able to intelligently predict which of these grasps are actually valid for a given object within a specific environment. By leveraging the reasoning capabilities of small Large Language Models (LLMs) [19, 20] and grounding them in physical simulation data, the pipeline can anticipate valid interactions before the user even makes contact. Ultimately, this research investigates how combining these two elements—metric-based intent classification and multimodal grasp prediction—can fundamentally enhance the XR user experience. To provide a complete evaluation of these objectives, the proposed GRIP-VR framework is rigorously benchmarked both quantitatively in terms of tracking accuracy and latency, and qualitatively against state-of-the-art Task-Oriented Grasping (TOG)

| Opp:<br>VF:    | Power                      |                                                                                                |          |                   |                                           |                 | Intermediate                            |                    | Precision            |                                                         |                                     |                                      |                                                                     |                    |
|----------------|----------------------------|------------------------------------------------------------------------------------------------|----------|-------------------|-------------------------------------------|-----------------|-----------------------------------------|--------------------|----------------------|---------------------------------------------------------|-------------------------------------|--------------------------------------|---------------------------------------------------------------------|--------------------|
|                | Palm                       |                                                                                                | Pad      |                   |                                           |                 | Side                                    |                    | Pad                  |                                                         | Side                                |                                      |                                                                     |                    |
|                | 3-5                        | 2-5                                                                                            | 2        | 2-3               | 2-4                                       | 2-5             | 2                                       | 3                  | 3-4                  | 2                                                       | 2-3                                 | 2-4                                  | 2-5                                                                 | 3                  |
| Thumb Abducted |                            | 1: Large Diameter<br>2: Small Diameter<br>3: Medium Wrap<br>10: Power Disk<br>11: Power Sphere | 31: Ring | 28: Sphere Finger | 18: Extension Type<br>26: Sphere 4-Finger | 19: Distal Type | 23: Adduction Grip                      |                    | 21: Tripod Variation | 9: Palmar Pinch<br>24: Tip Pinch<br>33: Inferior Pincer | 8: Prismatic 2 Finger<br>14: Tripod | 7: Prismatic 3 Finger<br>27: Quadpod | 6: Prismatic 4 Finger<br>12: Precision Disk<br>13: Precision Sphere | 20: Writing Tripod |
|                |                            |                                                                                                |          |                   |                                           |                 |                                         |                    |                      |                                                         |                                     |                                      |                                                                     |                    |
|                |                            |                                                                                                |          |                   |                                           |                 |                                         |                    |                      |                                                         |                                     |                                      |                                                                     |                    |
|                |                            |                                                                                                |          |                   |                                           |                 |                                         |                    |                      |                                                         |                                     |                                      |                                                                     |                    |
|                |                            |                                                                                                |          |                   |                                           |                 |                                         |                    |                      |                                                         |                                     |                                      |                                                                     |                    |
| Thumb Adducted | 17: Index Finger Extension | 4: Adducted Thumb<br>5: Light Tool<br>15: Fixed Hook<br>30: Palmar                             |          |                   |                                           |                 | 16: Lateral<br>29: Stick<br>32: Ventral | 25: Lateral Tripod |                      |                                                         |                                     |                                      | 22: Parallel Extension                                              |                    |
|                |                            |                                                                                                |          |                   |                                           |                 |                                         |                    |                      |                                                         |                                     |                                      |                                                                     |                    |
|                |                            |                                                                                                |          |                   |                                           |                 |                                         |                    |                      |                                                         |                                     |                                      |                                                                     |                    |
|                |                            |                                                                                                |          |                   |                                           |                 |                                         |                    |                      |                                                         |                                     |                                      |                                                                     |                    |
|                |                            |                                                                                                |          |                   |                                           |                 |                                         |                    |                      |                                                         |                                     |                                      |                                                                     |                    |

Figure 1.1: The Grasp Type Taxonomy proposed by Thomas Feix, outlining the 33 human grasp configurations (reproduced from [1]).

models such as DexTOG [12].

The research is guided by the following primary questions:

- RQ1:** How can the discrete grasp types of the Feix Taxonomy [1] be parametrically defined and categorized using real-time hand tracking metrics (e.g., joint angles and spatial hand data areas) to accurately classify varying grasp dimensions?
- RQ2:** How can a small Large Language Model (LLM) be effectively fine-tuned to predict valid robotic grasp types using multimodal inputs such as 3D object embeddings, spatial constraints, and object scale?
- RQ3:** To what extent does classifying the user’s hand tracking intent, rather than enforcing raw kinematic positions, improve the stability and visual fidelity of virtual grasping?

### 1.3 Proposed Approach

To address these challenges, this work proposes GRIP-VR (Grasp Recognition for Intention Prediction in Virtual Reality), an end-to-end pipeline divided into a predictive generative backend and an optimized Extended Reality (XR) frontend. The overall conceptual architecture and execution sequence of the GRIP-VR framework are illustrated in Figure 1.2.

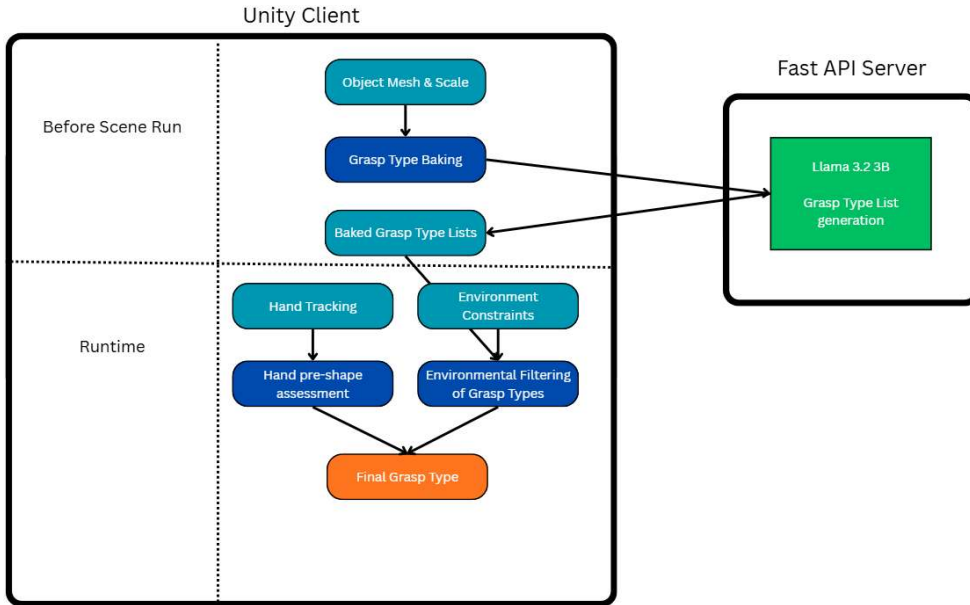


Figure 1.2: Conceptual architecture and system execution sequence of the GRIP-VR framework.

The foundation of this system relies on a rigorous data generation and analysis phase, where evaluation metrics for every grasp type within the Feix Taxonomy are computed using MuJoCo physics simulations and the Dexonomy dataset [3]. By applying random spatial constraints to various objects and evaluating the corresponding Dexonomy poses, a comprehensive dataset of valid grasp configurations is constructed. This data fuels a custom, parameter-efficient fine-tuning pipeline using Low-Rank Adaptation (LoRA) [11] for a small Large Language Model (LLM)—specifically the Meta LLaMA 3.2 3B Instruct model. Operating as a preliminary environmental filter, the LLM accepts multimodal inputs: 3D object embeddings generated via the OpenShape library [6], scene constraints, and object scale. To bridge the modality gap, a custom Multi-Layer Perceptron (MLP) projector maps the 1280-dimensional OpenShape embedding into a sequence of four virtual tokens spliced directly into the LLM’s vocabulary space. This enables the model to predict a subset of contextually valid grasp types for a given object, entirely independent of the user’s hand geometry.

Because this generative model acts strictly as an initial scene-level filter, its inference can be efficiently hosted on a lightweight computer running a Python FastAPI server, decoupled from the VR rendering engine. To ensure the high frame rates required for immersive XR, the Unity frontend employs a pre-computation, or “baking,” architecture. Before the virtual scene fully initializes, the system sends requests to the FastAPI backend to retrieve and store the LLM’s predicted list of valid grasp types for every interactable object based on its initial constraints. This architectural decision successfully shifts the heavy computational burden of multimodal reasoning out of the runtime loop.

At runtime, as the user’s physical hand approaches an object, the raw ego-centric optical hand tracking is first stabilized using a velocity-damped prediction heuristic to combat sensor noise and self-occlusion. GRIP-VR then relies on the previously established Feix Taxonomy metrics to dynamically classify the user’s hand orientation and finger configuration. Rather than attempting to generate a novel continuous pose, this runtime heuristic identifies the discrete grasp type the user is attempting to assume.

By cross-referencing this real-time spatial classification against the pre-baked, LLM-validated list of grasp types for that specific object, the system rapidly determines the most contextually appropriate interaction using penalty weights optimized through Differential Evolution. This intersection of real-time morphological tracking and pre-computed geometric validation efficiently deduces the user’s underlying intention, completely bypassing the computational overhead of dynamic pose generation.

## 1.4 Contributions of this Work

The primary contributions of this thesis are:

- Formulation and classification of the 33 distinct grasp types of the Feix Grasp

Taxonomy [1] using real-time egocentric hand tracking metrics, establishing a method for discrete user intent categorization during spatial interactions.

- Curation and engineering of a multimodal grasping dataset consisting of 140,000 unique simulated samples generated from 985 object meshes, correlating physics-validated grasps with 3D OpenShape [6] embeddings, scene constraints, and scale parameters.
- A parameter-efficient fine-tuning methodology utilizing Low-Rank Adaptation (LoRA) [11] for a lightweight Large Language Model (Meta LLaMA 3.2 3B Instruct) paired with a projection network mapping continuous 3D embeddings to standard grasp labels.
- Design and implementation of a decoupled client-server spatial computing architecture that shifts heavy model inference to the server via pre-computation (“baking”) and resolves dynamic scene constraints under 1 millisecond using local set operations. Empirical results demonstrate a grasp classification precision of 80.45% and a runtime constraint resolution latency of under 1ms, satisfying strict real-time XR frame budgets.

## 1.5 Thesis Structure

The remainder of this thesis is structured as follows. Chapter 2 reviews the state of the art in robotic grasping taxonomy, multimodal generative models, and XR hand tracking. Chapter 3 details the methodology behind dataset extraction and physics simulation using MuJoCo. Chapter 4 outlines the architecture and fine-tuning process of the multimodal LLM. Chapter 5 presents the implementation of the Unity XR frontend, detailing the baking process and the runtime constraint resolution algorithm. Chapter 6 evaluates the system, analyzing both the generative accuracy of the model and the computational performance of the XR pipeline. Finally, Chapter 7 discusses system limitations, provides interpretations of the results, and outlines directions for future research.

## Chapter 2

# Background and Related Work

### 2.1 Robotic Grasping and Taxonomy

Robotic dexterous grasping remains a fundamental challenge in enabling general-purpose embodied artificial intelligence. To successfully manipulate a wide variety of objects in unstructured environments, robotic systems must be capable of generating diverse, stable, and context-aware grasps. While parallel-jaw grippers have been extensively studied and successfully deployed, multi-fingered dexterous grasping is significantly more complex due to the high degrees of freedom (DoF) involved and the intricate contact dynamics required to maintain stability without causing object penetration [10]. Understanding and categorizing how human hands interact with objects provides the necessary theoretical foundation for mapping these complex biological capabilities to robotic end-effectors.

To address the sheer computational and mechanical complexity of dexterous manipulation, we formalize a three-tiered hierarchy of abstraction for virtual interaction: **Task Intention**  $\rightarrow$  **Grasp Type**  $\rightarrow$  **Continuous Joint Angles**.

1. **Task Intention:** At the highest level, this represents the cognitive and semantic goal of the interaction—what the user intends to do with the object (e.g., “drinking from a mug” or “using a knife to cut”).
2. **Continuous Joint Angles:** At the lowest level, this represents the continuous, high-dimensional kinematic space of the hand, consisting of the exact degrees of freedom (DoF) for every joint and skeletal node.
3. **Grasp Type:** Positioned in the middle, this represents the discrete, taxonomic classification of the hand shape (such as those defined by the Feix taxonomy [1]).

Mapping a user’s high-level Task Intention directly to Continuous Joint Angles in real-time is a mathematically ill-posed, highly noisy problem. In egocentric virtual reality, raw hand tracking data is inherently plagued by sensor noise and occlusions, making direct kinematic mapping extremely fragile. Conversely, Grasp

Types serve as an optimal semantic compression sweet-spot. By predicting the discrete Grasp Type from early approach trajectories, the system establishes a robust, hardware-agnostic representation of user intent. This intermediate classification effectively filters out sensor noise and isolates the high-level intent, which can then be mapped down to specific hand joint models or robotic end-effectors offline or via lightweight local heuristics, completely bypassing the need to dynamically resolve high-DoF collision and physics constraints in the active frame-rendering loop.

### 2.1.1 Foundational Grasp Taxonomies

Early attempts to classify human grasps were driven by the need to model hand kinematics for industrial applications. The widely recognized taxonomy by Cutkosky [21] primarily focused on manufacturing tasks, systematically observing machinists working with tools and metal parts. Cutkosky’s hierarchy broadly divided grasps into power grasps (where the object is clamped against the palm for maximum stability and force) and precision grasps (where the object is manipulated primarily with the fingertips for high dexterity).

Building upon these foundational works, the GRASP taxonomy proposed by Feix et al. [1] represents one of the most comprehensive and systematic categorizations of human grasping to date. By analyzing, comparing, and synthesizing decades of previous literature, the Feix taxonomy unified disparate terminologies to identify 33 distinct static, stable grasps performed by a single human hand.

The Feix taxonomy systematically arranges these grasp types based on four primary criteria, with the thumb alignment criteria illustrated in Figure 2.1:

- **Opposition type:** The anatomical surfaces applying opposing forces (e.g., pad, palm, or side opposition).
- **Virtual finger assignments:** The functional grouping of biological digits acting as a single cohesive unit to apply force against an object.
- **Grasp nature:** The continuum of the grasp in terms of power, precision, or an intermediate state.
- **Thumb position:** The specific positional alignment (adducted or abducted) of the thumb relative to the fingers.

This standardized terminology is critical for robotics, as it provides a structured framework to evaluate grasp diversity and serves as a blueprint for synthesizing human-like poses. By breaking down complex hand-object interactions into quantifiable parameters—such as finger curl and opposition profiles—the taxonomy allows researchers to cluster grasp types and generate dimensional landscapes that map the similarities and structural relationships between different grasping strategies [1].

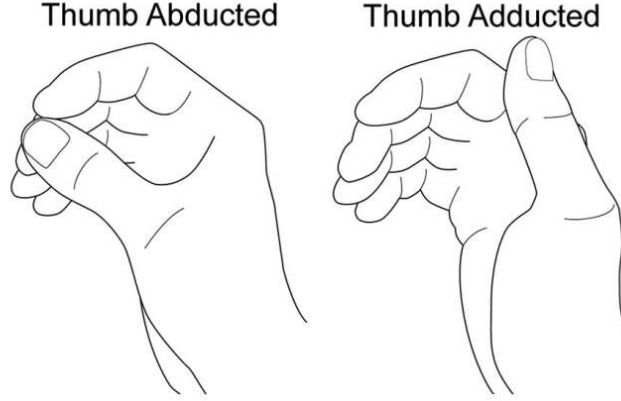


Figure 2.1: The thumb position criteria (adduction vs. abduction) used in the Feix Grasp Taxonomy (reproduced from [1]).

### 2.1.2 Simulation and Large-Scale Grasp Synthesis

While taxonomies provide the theoretical framework for dexterous manipulation, training modern data-driven robotic models requires massive, high-quality datasets. Collecting human demonstration data via motion capture (mocap) is notoriously expensive and difficult to scale. Consequently, the field has heavily shifted toward simulation-based grasp synthesis. Datasets such as DexGraspNet [10] spearheaded this transition by utilizing advanced physics simulators to generate over 1.3 million stable grasps for highly articulated robotic hands across thousands of objects. To overcome the computational bottlenecks of simulating millions of interactions, subsequent works like BODex [2] (examples of which are shown in Figure 2.2) introduced bilevel optimization frameworks, dramatically improving the scalability and efficiency of synthesizing stable robotic grasps.



Figure 2.2: Visual examples of multi-fingered grasps generated in the BODex dataset (reproduced from [2]).

Despite these advancements in scale, many purely optimization-driven datasets suffer from a lack of taxonomic diversity, often collapsing into a narrow subset of structurally similar power or pinch grasps [3]. The Dexonomy dataset [3] directly addresses this critical bottleneck by providing a massive collection of contact-rich, penetration-free grasps that explicitly and evenly cover the 33 categories defined within the Feix taxonomy. Unlike prior automatic synthesis methods that are frequently limited to specific object categories or generic stability metrics, Dexonomy offers an expansive, automated exploration of the entire taxonomic space, complete with learning-based grasp synthesis baselines (such as diffusion models and normalizing flows) to map these geometries [22].



Figure 2.3: Visual examples of taxonomically complete hand poses from the Dexonomy dataset (reproduced from [3]).

### 2.1.3 Bridging Human Priors and Robotic Kinematics

Translating these theoretical and simulated grasp types into physical robotic applications requires an end-effector capable of high dexterity. In robotics, an end-effector is the physical device or tool situated at the distal end of a robotic arm that directly interacts with the environment—serving the same structural and functional purpose as the human hand. A prominent target platform for this mapping is the Shadow Dexterous Hand [23], an advanced, highly anthropomorphic robotic hand utilizing 24 degrees of freedom (DoF) to closely mirror human skeletal structure and joint movement ranges. The Shadow Hand’s high degrees of freedom make it an ideal target platform for reproducing the Feix taxonomy grasps evaluated in the Dexonomy dataset [3] (as illustrated in Figure 2.3).

However, recent literature indicates that achieving mathematical grasp stability (e.g., force closure) is insufficient for seamless integration into human-centric environments or spatial computing. Grasps must also be affordance-aware and

contextually natural. Frameworks like AffordDex [24] highlight that neglecting human-like positioning priors often results in robotic grasps that, while stable, block the functional parts of an object (e.g., grasping a mug by its opening rather than its handle). To bridge this gap, datasets like DexCanvas [25] have begun hybridizing real human demonstrations with synthetic robotic learning, utilizing thousands of hours of mocap data to ensure that simulated multi-fingered grasps obey human behavioral priors.

By analyzing collision matrices derived from grasp type metrics and dynamically evaluating grasp poses against environmental constraints, the kinematic profiles of simulated hands can be optimized to successfully execute the full spectrum of taxonomical grasps [3]. For immersive Extended Reality (XR), this wealth of synthetic, taxonomically complete data provides the exact parametric boundaries needed to classify a human user’s noisy hand tracking data into a discrete, affordance-aware intent.

## 2.2 Generative AI and Multimodal Learning

The integration of generative artificial intelligence and multimodal learning has profoundly transformed robotic perception and manipulation. Biological systems process the world by simultaneously interpreting high-dimensional inputs from diverse modalities, such as vision, audition, and proprioception. Replicating this in artificial systems has historically been hindered by the need for domain-specific network architectures. However, foundational architectures like the Perceiver have demonstrated that iterative attention mechanisms can effectively handle these asymmetric, multi-modal inputs without relying on rigid, domain-specific grid structures [26]. Concurrently, visual language models (VLMs) such as Flamingo have proven that large-scale models can be rapidly adapted to novel tasks using only a handful of annotated examples, successfully bridging pretrained vision-only and language-only models to process interleaved visual and textual data [4]. The architectural structure of the Flamingo VLM is depicted in Figure 2.4.

### 2.2.1 Large Language Models as Planners and Policy Generators

Within embodied AI, Large Language Models (LLMs)—and their more efficient, smaller-scale counterparts—are increasingly utilized not just for linguistic tasks, but as zero-shot planners capable of extracting actionable knowledge. Without any environment-specific training, these models can effectively decompose high-level natural language instructions into sequences of low-level, admissible robotic actions [19].

However, generating executable plans in physical environments requires models to account for real-world constraints. Frameworks such as SayCan ground language models in robotic affordances by assessing both what the model determines is the most useful action and what the robot’s low-level policies indicate is physically possible within a given scene [27]. Extending beyond planning, LLMs

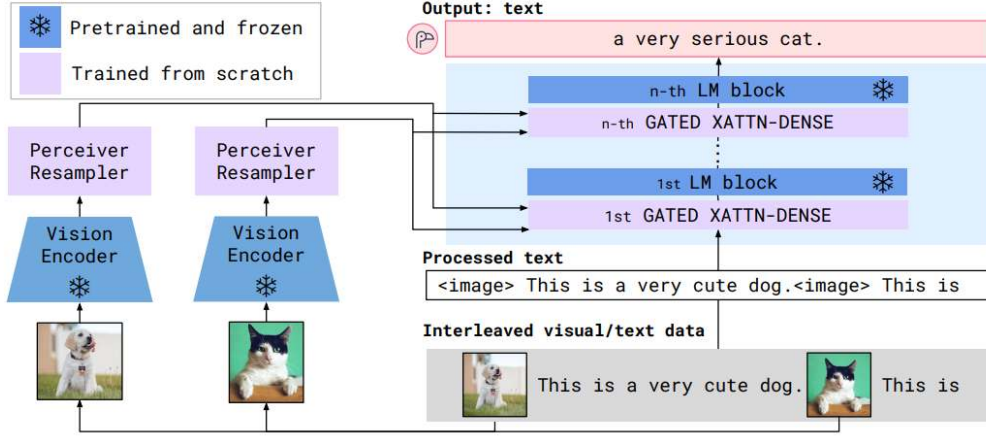


Figure 2.4: Structural overview of the Flamingo visual language model (reproduced from [4]).

initially trained on code completion have been successfully repurposed to synthesize robot policy code directly from natural language commands. Frameworks like Code as Policies demonstrate that LLMs can express complex feedback loops that parameterize control primitives and process real-time perception outputs [28].

Taking this a step further into direct physical control, Vision-Language-Action (VLA) models, such as RT-1 [5], have demonstrated that transformer architectures can absorb large-scale robot demonstration data to directly output discrete, tokenized robotic actions from raw sensory and language inputs. This end-to-end action mapping is demonstrated in the RT-1 architecture (see Figure 2.5).

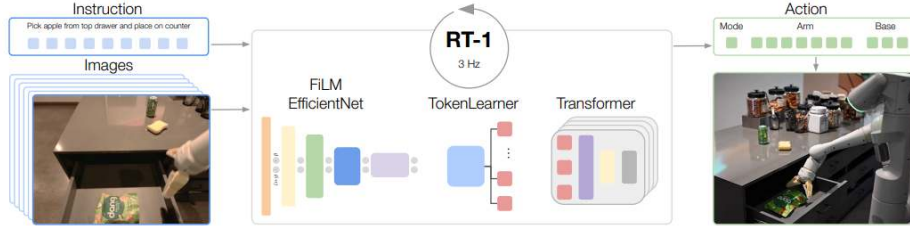


Figure 2.5: The Robotics Transformer (RT-1) architecture mapping vision and language inputs to tokenized control primitives (reproduced from [5]).

To adapt these massive generative architectures for specialized, real-time classification tasks (such as grasp type prediction) without the computational burden of full retraining, Parameter-Efficient Fine-Tuning (PEFT) methods have become standard. Techniques like Low-Rank Adaptation (LoRA) [11] freeze the pre-trained model weights and inject small, trainable rank decomposition matrices into the transformer layers. This allows LLMs to be rapidly optimized for discrete

spatial reasoning tasks while minimizing hardware overhead, making them ideal for deployment in decoupled XR architectures.

### 2.2.2 3D Object Representation and Affordance Grounding

While traditional LLMs and VLMs excel at commonsense reasoning, they inherently lack grounding in the 3D physical world, which is governed by spatial relationships, physics, and complex object geometries. Injecting 3D representations into these models is necessary to perform holistic scene understanding and affordance reasoning. Recent frameworks like 3D-LLM [20] directly address this gap by injecting 3D point clouds and their features into large language models, enabling them to navigate and reason about spatial layouts and physical affordances.

A critical advancement in extracting these geometries is OpenShape, a method designed to scale up 3D shape representations for open-world understanding [6]. By ensembling multiple massive 3D datasets, OpenShape utilizes a multi-modal contrastive learning framework to align 3D point cloud encoders with established 2D image and language embedding spaces, such as CLIP. The retrieval capabilities of this aligned embedding space are shown in Figure 2.6.



Figure 2.6: Representation of multimodal joint embedding space in the OpenShape framework (reproduced from [6]).

This creates a highly generalizable joint representation space encompassing text, 2D images, and 3D shapes. For robotic dexterous grasping, leveraging such comprehensive 3D embeddings is crucial, as it allows systems to classify novel object geometries in a zero-shot manner and condition grasp synthesis models on rich, semantic, and structurally accurate 3D representations.

Furthermore, this 3D semantic understanding is intrinsically tied to the concept of affordances—the actionable properties of an object. Early works demonstrated that modeling an object’s affordance (e.g., how a human would grasp it) significantly improves robust object recognition [29]. Modern deep learning approaches, such as AffordanceNet [30], have evolved to simultaneously detect multiple objects and assign pixel-level affordance labels, identifying precisely where an object

should be grasped or manipulated. Expanding this to dynamic interactions, recent research leverages internet videos of human behavior to extract versatile, environment-centric affordance representations [31]. By integrating these affordance priors with 3D object embeddings, multimodal LLMs can act as highly effective preliminary filters, deducing not just if an object can be grasped, but how it should functionally be interacted with in a virtual environment.

### 2.2.3 Semantic Intention Prediction and Task-Oriented Grasping

While foundational taxonomies define how a hand poses, recent advancements in Embodied AI have attempted to address why a grasp is chosen by predicting semantic user intention. Task-Oriented Grasping (TOG) systems, such as GraspGPT [32] and SemGrasp [7] (shown in Figure 2.7), leverage the reasoning capabilities of large Vision-Language Models (VLMs) to align 3D point clouds and object representations with human language instructions. By processing natural language commands alongside complex scene geometries, these systems successfully map user intent to specific object affordances.

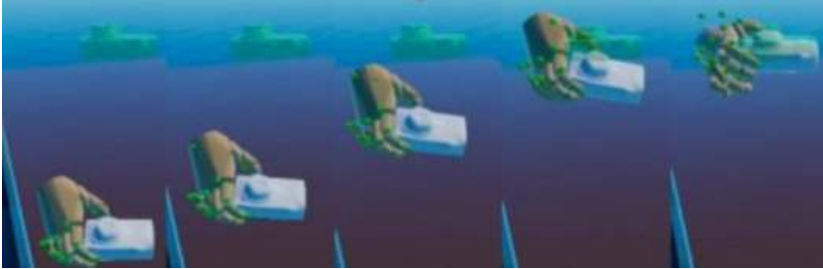


Figure 2.7: Semantic grasping interaction interface within a virtual reality environment (reproduced from [7]).

Similarly, recent frameworks like DexTOG [12] utilize learning-based synthesis to generate highly accurate, task-specific dexterous hand poses based on integrated text and 3D mesh inputs. However, while these VLM-driven systems excel at offline semantic reasoning and pose generation, they suffer from severe computational overhead. Processing raw pixels or voxels through a multimodal architecture introduces significant latency that is entirely unsuitable for real-time 90Hz Virtual Reality environments. Furthermore, purely visual models are prone to hallucinations when confronted with occlusion. Consequently, there remains a critical gap in the literature for a system that can achieve the deep semantic understanding of a VLM without violating the strict motion-to-photon latency budgets required by spatial computing.

## 2.3 Spatial Computing and Hand Tracking

### 2.3.1 XR Hand Tracking Capabilities and Limitations

The integration of spatial computing in robotics and virtual environments heavily relies on accurate hand tracking to facilitate natural, uninstrumented user interactions. Modern Extended Reality (XR) systems typically utilize outward-facing cameras coupled with machine learning pipelines to estimate the 3D skeletal pose of the user’s hands in real-time. Despite advancements in these tracking frameworks, vision-based approaches face persistent physiological and technical limitations. The most prominent challenges include severe self-occlusion—where fingers obscure one another or the palm from the camera’s viewpoint during a grasp—and field-of-view restrictions that instantly lose track of the hands when they drop out of sight or move too rapidly.

Furthermore, biological grasping is highly nuanced, involving complex hand configurations and soft tissue deformations that create intricate regions of contact between the hand and the object. Capturing and accurately modeling this physical contact purely through optical tracking remains exceptionally difficult. This gap has driven the need for specialized datasets, such as ContactPose [8] (examples of which are shown in Figure 2.8), which pair visual modalities with precise, high-resolution contact metrics to capture the actual physical interplay between human skin and object surfaces. Moving beyond static, isolated grasps, human grasping is fundamentally a coordinated, whole-body action. Datasets like GRAB [33] have demonstrated that modeling realistic human grasping requires capturing the full 3D shape and pose sequences of the entire body moving over time, highlighting the sheer dimensionality required to perfectly simulate human interaction.

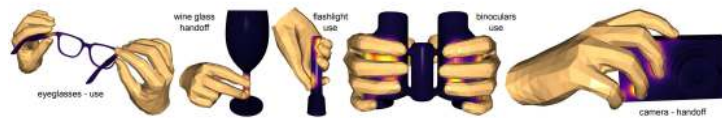


Figure 2.8: Optimization of MANO hand models aligned with physical object contact heatmaps in ContactPose (reproduced from [8]).

To bridge the gap between these high-fidelity laboratory datasets and real-world spatial computing, researchers have heavily focused on egocentric (first-person) interactions, mirroring the perspective of a VR headset. Massive datasets like HOI4D [34] provide 4D egocentric streams of humans interacting with diverse, category-level objects in the wild. These datasets establish a critical link between dynamic hand poses and functional interaction intentions from a first-person perspective, providing the rich temporal data necessary to train spatial computing algorithms.

### 2.3.2 Semantic Affordances and Multi-Object Context

Accurate hand kinematics alone do not solve the problem of virtual interaction; the system must also understand why the user is moving their hand. Hand-object interaction is inherently driven by object affordances. A user’s grip changes drastically depending on whether they intend to pass a mug, drink from it, or wash it. Frameworks like OakInk [35] function as multi-modal knowledge repositories, bridging visual understanding with the cognitive intent behind hand-object interactions. Similarly, AffordPose [9] maps large-scale hand-object interactions specifically to part-level affordance labels, proving that grasping is intrinsically dictated by the functional role of the target object. The construction pipeline of the AffordPose dataset linking hand shapes to functional object regions is shown in Figure 2.9.

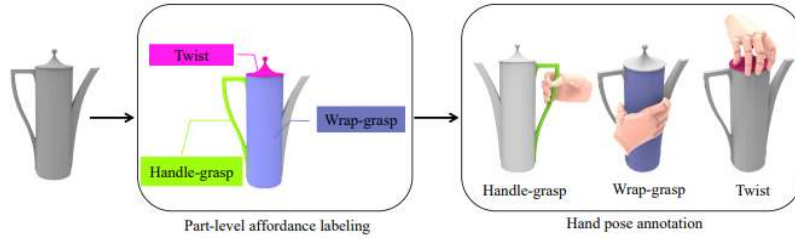


Figure 2.9: The Affordpose dataset construction process (reproduced from [9]).

In complex, multi-object XR environments, predicting this affordance-driven intent becomes computationally daunting. Generative models like GanHand [36] have explored this by predicting human grasp affordances across scenes with multiple objects from a single perspective, synthesizing physically plausible 3D hand shapes that minimize graspability loss. These works collectively prove that generating realistic virtual interactions requires a deep, semantic understanding of the scene’s geometric structure, rather than just isolated hand tracking.

### 2.3.3 Pose versus Intent and VR Latency Constraints

A fundamental challenge in spatial computing is the distinction between raw hand pose (the spatial coordinates and joint angles) and user intent (the functional, semantic goal of the user’s movement). While current XR tracking APIs successfully capture kinematic pose, they do not inherently provide any semantic understanding of the interaction. This gap is particularly problematic in interactive environments where point-to-point processing and physics rendering latency can severely disrupt the user experience. Human-computer interaction research establishes that end-to-end motion-to-photon latency in immersive VR must strictly remain minimal; latencies exceeding standard thresholds rapidly induce cybersickness and drastically degrade human motor performance [17]. Attempting to continuously resolve high-degree-of-freedom, collision-free joint angles dynamically places immense strain on this critical latency budget, often resulting in visual penetration

or physics engine failures.

To bridge the gap between reactive pose estimation and proactive interaction, the paradigm is shifting from mapping “Operation-to-Intent” toward “Intent-to-Operation” frameworks [37]. Recent research focuses on predicting user intent directly from early hand kinematics during the initial reach-to-grasp action, completely bypassing the need to render continuous physical collisions. By analyzing local finger motion dynamics as a user’s hand approaches a target, systems can achieve real-time recognition of the intended uninstrumented object. For example, sequential deep learning models, such as LSTM networks, have demonstrated the ability to process these kinematic sequences to predict the exact moment a user will grasp an object with a precision of under 21 milliseconds, while also estimating the distance to the target with sub-centimeter accuracy before contact is even made [18].

## Chapter 3

# Dataset Engineering and Processing

The foundation of the proposed predictive framework relies on a rigorous data generation and evaluation phase. To train a multimodal Large Language Model (LLM) to accurately predict grasp types, it is first necessary to construct a comprehensive, physics-validated dataset that maps object geometries and spatial constraints to successful robotic grasps. This chapter details the extraction, simulation, and curation pipeline used to process raw pose data into a structured format suitable for generative fine-tuning.

### 3.1 Physics Simulation and Evaluation Setup

The primary source of grasp data for this research is the Dexonomy dataset, which provides an expansive collection of hand poses categorized across the 33 grasp types of the Feix taxonomy. The objects within this dataset are sourced from the DexGraspNet (DGN) and Objaverse datasets, providing a rich variety of real-world scanned geometries. While the generative algorithm used to originally synthesize these poses remains unreleased, the resulting static poses are publicly available.

To determine which of these static poses represent physically viable grasps, the system utilizes DexGraspBench, a specialized benchmarking tool designed to evaluate, analyze, and visualize multi-fingered hand poses. DexGraspBench integrates directly with the MuJoCo physics engine to simulate the physical interaction between the robotic end-effector (the Shadow Hand) and the target object (as illustrated in the MuJoCo simulation in Figure 3.1). During simulation, external forces are applied to the object while the simulated hand attempts to maintain its grasp; if the object slips or the grasp fails to counteract the applied forces, the pose is deemed unsuccessful.



Figure 3.1: Physics-based multi-fingered grasp simulation executed inside the MuJoCo engine (reproduced from [10]).

### 3.1.1 Optimization of the Evaluation Pipeline

The original evaluation scripts provided by DexGraspBench were highly unoptimized for large-scale dataset processing. Initial testing revealed that evaluating the complete Dexonomy dataset would take well over a week to execute. Profiling the execution pipeline highlighted a severe disk Input/Output (I/O) bottleneck. The original structure stored individual hand poses in separate files, forcing the evaluation script to perform thousands of fragmented read operations. Despite utilizing high-performance hardware, the sheer volume of file requests throttled the physics simulation.

To resolve this, a custom data processing architecture was implemented. First, the format conversion scripts were heavily modified to consolidate the scattered hand pose data. Instead of maintaining separate files, all hand poses for every grasp type associated with a specific object were aggregated into a single, highly optimized `.npy` file. This restructuring drastically reduced I/O overhead, allowing the physics engine to load an entire object’s grasp array into memory simultaneously.

Second, the evaluation script itself was streamlined. The original benchmarking tool computed an exhaustive list of analytics for every pose. To further reduce the computational workload, the script was modified to calculate and output only the most critical indicators of grasp stability:

1. **Success Rate:** The binary outcome of the MuJoCo physics simulation under applied forces.

2. **Average DFC (Distance to Force Closure):** A metric indicating how close the grasp is to achieving a mathematically perfect force closure.
3. **Average Delta Angle:** The kinematic deviation between the initial predicted joint angles and the final stable joint angles after the physics simulation settles.

Specifically, a grasp is classified as physically successful (Success Rate = 1) if the average Distance to Force Closure (DFC) remains below a threshold of 0.05 and the Average Delta Angle (representing kinematic settling deviation) is less than  $15^\circ$  under the applied perturbation forces in MuJoCo. While these continuous metrics (DFC and Delta Angle) are critical for establishing the initial ground-truth filtering during this simulation phase, they are not reported in the downstream runtime evaluation since they are abstracted away into the binary success label that defines the valid grasp type lists.

These optimizations successfully reduced the total dataset evaluation time from over a week to approximately 1 day. The culmination of this simulated evaluation phase is the generation of a structured `eval.json` file for every object scale, cataloging these specific stability metrics for each taxonomical grasp type.

## 3.2 Data Extraction Pipeline

With the physics evaluation complete, the next phase of the pipeline involves extracting and fusing the physical metrics and object geometries into a unified format that a multimodal model can interpret. Initially, this research explored extracting manual geometric metadata—such as the center of gravity, Oriented Bounding Boxes (OBB), and convex hull decompositions via CoACD and PCA—to describe the object’s shape. However, it quickly became apparent that these handcrafted heuristics lacked the fidelity required to accurately represent complex, non-convex meshes, leading to a loss of critical geometric affordances.

To overcome this limitation, the pipeline was redesigned to utilize direct semantic representations of the 3D meshes. Leveraging the OpenShape framework [6], the original `.obj` files from the Dexonomy and DGN datasets are processed to generate dense 3D object embeddings. OpenShape successfully encodes the complete geometric and semantic structure of the unaligned 3D mesh into a high-dimensional vector space that aligns with standard language and vision modalities. This ensures that the generative model receives a highly accurate, holistic representation of the object’s true geometry rather than a manually approximated bounding box.

In addition to the geometric embedding, a virtual interaction pipeline must account for spatial context and size. First, the specific geometric scale of the object is extracted alongside the mesh, as a single object (e.g., a generic cylinder) requires entirely different grasp types depending on whether it is scaled to the size of a pill bottle or a barrel.

Second, the system introduces simulated scene constraints. In real-world and VR environments, objects are rarely floating freely in space; they rest on tables, lean against walls, or fall sideways on the floor. To represent this mathematically, the pipeline generates constraint parameters that selectively block specific spatial axes (e.g., blocking the negative Y-axis to simulate an object resting flat on a table). These three elements—the OpenShape embedding, the object scale, and the blocked axis constraints—form the complete multimodal input required for the predictive model.

### 3.3 Dataset Curation and Formatting for LLM Fine-Tuning

To fine-tune a Large Language Model (LLM) to act as a discrete grasp classifier, the extracted modalities and simulation metrics must be curated into a structured, supervised instruction dataset.

To construct the dataset, 140,000 unique samples were generated from 985 different object meshes sourced from the DexGraspNet (DGN) and Objaverse datasets. To ensure robust generalization and prevent shape-level data leakage during model evaluation, a strict shape-wise partition was implemented with an 80/20 split: 788 meshes were allocated exclusively to the training set, while the remaining 197 meshes were reserved for the validation set. This partition yielded a training set of approximately 112,000 samples (from 788 meshes) and a validation set of 28,000 samples (from 197 meshes). This partition ensures that the model is evaluated on completely unseen object geometries. By applying different axis constraints across these meshes, the dataset comprehensively maps the relationship between the multimodal inputs and the valid grasp types, ensuring that the 33 categories of the Feix Grasp Taxonomy are evenly balanced. Each individual sample in the fine-tuning dataset is structured with the following components (an example of a compiled training sample is shown in Figure 3.2):

1. **Inputs:** The file path to the pre-computed OpenShape object embedding, the specific scale value, and the defined scene constraints (the specific axes that are blocked for that instance).
2. **System Prompt:** A refined instruction prompt dictating the model’s behavior.
3. **Expected Output:** The ground-truth array of valid grasp type IDs.

To optimize the LLM for high-frequency XR applications, the system prompt was engineered for strict token efficiency. The prompt explicitly instructs the LLM to output only a raw list of valid grasp type IDs corresponding to the 33 categories of the Feix Taxonomy [1] (e.g., [3, 12, 24]), stripping away any conversational filler or chain-of-thought text. This design significantly reduces the number of

```

{
  "embeddings": [
    "embeddings/objaverse/26a9e577a2314750a913912baae3cc57/embedding.npy"
  ],
  "conversations": [
    {
      "from": "human",
      "value": "<embedding>\nOBJECT SCALE: 0.05\nSCENE CONSTRAINTS:\nBlocked
        axes (m):\n- -X axis\n- +Y axis\n- -Z axis\n- +Z axis"
    },
    {
      "from": "gpt",
      "value": "6, 16, 20"
    }
  ],
  "system": "You are a robotic grasping AI. Based on the 3D geometry provided,
    output a comma-separated list containing the most suitable grasp IDs.
    Allowed IDs: 1 to 33. Output only the comma-separated numbers and stop."
}

```

Figure 3.2: A sample from the dataset created for fine tuning a small LLM for GRIP-VR.

generated tokens, which directly accelerates inference speeds on the FastAPI server and simplifies the parsing logic required by the Unity client.

The ground-truth “Expected Output” for each sample is derived by cross-referencing the previously generated `eval.json` metrics with the imposed scene constraints. The script evaluates the physically successful grasp poses from the MuJoCo simulation and filters them against the blocked axes. If a mathematically stable grasp from `eval.json` requires the hand to approach from an axis that is currently blocked by the scene constraint (e.g., attempting to grasp the bottom of an object resting on a table), that grasp type is invalidated and removed from the expected output list.

### 3.3.1 Mathematical Representation of Constraint Blocking

To formalize the constraint logic, we define the spatial domain of cardinal approach directions in three-dimensional space as:

$$\mathcal{D} = \{+X, -X, +Y, -Y, +Z, -Z\} \quad (3.1)$$

Let  $G$  be the set of all stable grasps for an object derived from physical simulation. Each individual grasp  $g \in G$  requires a specific subset of approach directions to be physically accessible, represented as  $A(g) \subseteq \mathcal{D}$ .

When the object is placed in a virtual environment, scene constraints block a subset of these approach directions, represented as the constraint set  $C \subseteq \mathcal{D}$  (for instance,  $C = \{-Y\}$  when an object is resting flat on a table). A grasp  $g$

### 3.3. DATASET CURATION AND FORMATTING FOR LLM FINE-TUNING<sup>23</sup>

is physically invalidated if any of its necessary approach paths intersect with the environmental constraints. Thus, the resulting set of valid expected grasp outputs  $G_{valid}$  is calculated as:

$$G_{valid} = \{g \in G \mid A(g) \cap C = \emptyset\} \quad (3.2)$$

To prevent the model from overfitting to specific orientations and to ensure it genuinely learns the spatial relationship between the mesh and the environment, the dataset was rigorously balanced. For every object mesh included in the dataset, samples were generated covering every possible combination of axis constraints. This ensures the LLM learns a robust, generalized mapping between the 3D embedding, the environmental blockers, and the resulting taxonomically valid grasp types.

#### 3.3.2 Ethical and Privacy Considerations

To ensure compliance with academic and research ethics, it is noted that all hand-tracking joint data and 3D mesh geometries utilized for training the generative model are entirely synthetic and sourced from publicly available research repositories (Dexonomy and DexGraspNet). No human subject biometric data was collected, captured, or stored during the dataset engineering and training phases. For the qualitative user evaluation, live hand tracking coordinates processed by the Unity XR Hands API are evaluated locally and in real-time as abstract joint angles and are immediately discarded. No raw camera feeds, biometric identifiers, or persistent user motion profiles are captured, recorded, or transmitted, strictly adhering to user data privacy guidelines in spatial computing.

## Chapter 4

# Multimodal LLM Architecture

The predictive core of GRIP-VR relies on translating complex 3D geometric constraints into discrete grasp categories. This chapter details the architectural decisions, embedding strategies, and specialized training methodologies required to transform a standard text-based Large Language Model (LLM) into a multimodal, affordance-aware reasoning engine.

### 4.1 Model Selection and Baseline Architecture

Deploying generative models within Extended Reality (XR) architectures imposes strict computational constraints. The ultimate objective for spatial computing is to execute interaction logic entirely on-device (e.g., directly on a Meta Quest headset). However, current standalone VR hardware possesses severely limited Video RAM (VRAM) and processing overhead, making the local execution of highly parameterized LLMs currently unfeasible without inducing unacceptable rendering latency.

Consequently, the architecture requires a model small enough to be hosted efficiently on everyday consumer-grade machines acting as local servers, yet powerful enough to perform complex zero-shot reasoning. During our design and evaluation phase, we evaluated and compared several prominent open-weight architectures, specifically **Qwen2.5-3B-Instruct** [38] and **Phi-3.5-mini** [39], benchmarking their hardware footprints, context processing efficiency, and structured output adherence. Following these comparisons, the **Meta LLaMA 3.2 3B Instruct** model was selected as the foundational baseline. With 3 billion parameters, it occupies a highly efficient computational footprint—requiring approximately 6GB of VRAM when loaded in bfloat16 precision—making it easily deployable on standard desktop GPUs, while showing superior zero-shot instruct-following reliability for structured list generation. The structural block layout of the LLaMA 3.2 3B Instruct transformer architecture is illustrated in Figure 4.1.

Furthermore, the LLaMA 3.2 3B architecture utilizes a dense transformer configuration with a hidden dimension of 3072. Because the “Instruct” variant is

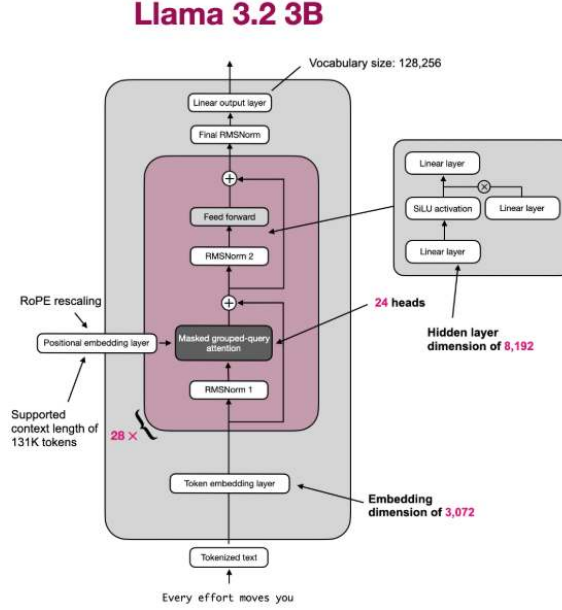


Figure 4.1: The LLaMA 3.2 3B Instruct transformer block architecture (reproduced from [11]).

heavily pre-trained on conversational and instructional data, it exhibits strong adherence to strict formatting constraints out of the box. This characteristic is critical for the pipeline, as the model must reliably output parseable arrays of numerical IDs rather than conversational text.

## 4.2 Input Modalities and Embedding Strategy

Standard LLMs are unimodal, designed exclusively to process discrete text tokens. To enable the LLaMA 3.2 model to reason about 3D object geometries, a custom continuous embedding strategy was engineered to fuse spatial data directly into the model’s textual latent space.

The physical geometry of the target object is represented by a 1280-dimensional OpenShape embedding. To bridge the dimensionality gap between the OpenShape encoder and the LLaMA 3.2 vocabulary space, a custom Multi-Layer Perceptron (MLP) mapping network, acting as a custom projector, was implemented.

Mathematically, let  $x \in \mathbb{R}^{1280}$  be the input OpenShape point-cloud embedding. The projector maps this vector to a sequence of 4 continuous virtual tokens, represented as a matrix  $\mathbf{V} \in \mathbb{R}^{4 \times 3072}$ . This is achieved through a sequence of linear transformations and non-linear activations:

$$h = \text{GELU}(W_1 x + b_1) \quad (4.1)$$

$$\mathbf{V} = \text{Reshape}(W_2 h + b_2, (4, 3072)) \quad (4.2)$$

where  $W_1 \in \mathbb{R}^{6144 \times 1280}$  and  $b_1 \in \mathbb{R}^{6144}$  represent the weights and bias of the first projection layer, which projects the embedding into a hidden space of 6144 dimensions. The second projection layer, parameterized by  $W_2 \in \mathbb{R}^{12288 \times 6144}$  and  $b_2 \in \mathbb{R}^{12288}$ , maps this hidden vector to the flat 12288 dimensions ( $4 \times 3072$ ) required to form the four separate 3072-dimensional sequence tokens. Both layers are initialized using normal weight distributions,  $W_1, W_2 \sim \mathcal{N}(0, 0.01^2)$ , with zero biases to ensure stable initial optimization gradients.

Crucially, compressing highly intricate 3D object topologies into a single vector often results in an information bottleneck. To provide the LLM with sufficient representational bandwidth, the projector expands the single OpenShape embedding into a sequence of four distinct “virtual tokens” (as illustrated in Figure 4.2).

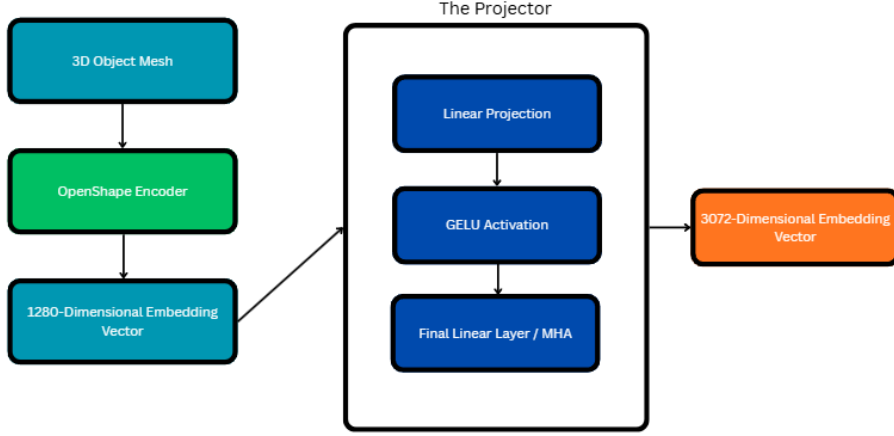


Figure 4.2: Schematic representation of OpenShape 3D embeddings projected into the 3072-dimensional space of LLaMA (reproduced from [6]).

Integrating these continuous virtual tokens requires overriding the standard forward pass of the LLM. During the tokenization phase, the textual prompt (containing the scene constraints and system instructions) is processed into standard integer IDs. This splicing is implemented seamlessly at the token embedding level. During dataset preprocessing and collation, four dummy placeholder tokens (initialized to index 0) are inserted into the input token ID sequence immediately following the start-of-sequence token (index 1).

At runtime, the model’s standard embedding layer converts the text IDs into continuous vectors, and the forward pass replaces the embeddings corresponding to these dummy tokens with the projected virtual tokens  $\mathbf{V}$ . Mathematically, if  $\mathbf{X}_{text} \in \mathbb{R}^{N \times 3072}$  represents the embedded sequence of  $N$  textual tokens, the final sequence  $\mathbf{X}_{input} \in \mathbb{R}^{(N+4) \times 3072}$  fed into the LLaMA transformer blocks is constructed as:

$$\mathbf{X}_{input} = \left[ \mathbf{X}_{text}[:, 0 : 1, :] \parallel \mathbf{V} \parallel \mathbf{X}_{text}[:, 5 : N, :] \right] \quad (4.3)$$

where  $\parallel$  denotes tensor concatenation along the sequence dimension, and the indices correspond to the replacement of the dummy tokens at indices 1 to 4. This allows the transformer’s attention mechanisms to attend to the 3D geometry seamlessly alongside the textual constraints.

### 4.3 Fine-Tuning Methodology

Training this multimodal architecture requires specialized optimization techniques to align the newly initialized projector network with the pre-trained knowledge of the LLM without degrading the model’s foundational reasoning capabilities.

Fully retraining a 3 billion parameter model is highly resource-intensive and prone to catastrophic forgetting. Instead, a Low-Rank Adaptation (LoRA) strategy was employed. The pre-trained weights of the LLaMA 3.2 model were frozen, and low-rank trainable matrices (with a rank  $r = 16$  and an alpha scaling factor of 32) were injected into the attention and feed-forward layers (`q_proj`, `k_proj`, `v_proj`, `o_proj`, `gate_proj`, `up_proj`, `down_proj`). This method reduces the trainable parameter count to a fraction of the full model size, enabling highly efficient optimization while preserving the core instruct-following capabilities.

The architecture presents a unique optimization challenge: the custom projector is initialized with randomized weights and possesses zero prior knowledge, whereas the LoRA weights are adapting a highly mature LLM. To address this, a “two-speed” optimization configuration was implemented using the AdamW optimizer. The projector parameters are updated aggressively with a learning rate of  $1 \times 10^{-3}$ , allowing the network to rapidly learn the mapping from OpenShape to the LLaMA latent space. Simultaneously, the LoRA parameters are updated at a much more conservative learning rate of  $2 \times 10^{-5}$ . A linear learning rate scheduler with a 10% warmup phase ensures stable convergence across both modalities.

Table 4.1 outlines the specific hyperparameters utilized during the training phase.

Standard language model training monitors optimization via cross-entropy loss, which measures next-token probability. However, low loss does not guarantee that the model will generate valid, properly formatted arrays of grasp taxonomy IDs during actual deployment. To strictly monitor task performance, a custom **GenerativeEvalCallback** was engineered. Periodically during the training cycle, the optimization loop pauses, and the model is forced into generation mode on a holdout validation set (the tracked metrics interface of which is shown in Figure 4.3). The callback dynamically generates predictions, strips away any conversational artifacts, and parses the output into numerical arrays. These predictions are then strictly evaluated against the ground-truth sets to calculate exact Precision, Recall, and F1-scores. Furthermore, the callback tracks the “Valid Format Rate” to ensure the model does not drift from the rigid JSON-style array structure required by the Unity XR client. This generative evaluation ensures the fine-tuning process directly optimizes for downstream interaction accuracy rather than mere

linguistic perplexity.

Table 4.1: Complete training hyperparameters for the GRIP-VR multimodal LLM.

| Hyperparameter          | Value                                                               |
|-------------------------|---------------------------------------------------------------------|
| Base Model              | Meta LLaMA 3.2 3B Instruct                                          |
| Compute Precision       | Brain Floating Point 16 (bfloat16)                                  |
| LoRA Target Modules     | q_proj, k_proj, v_proj,<br>o_proj, gate_proj, up_proj,<br>down_proj |
| LoRA Rank ( $r$ )       | 16                                                                  |
| LoRA Alpha ( $\alpha$ ) | 32                                                                  |
| LoRA Dropout            | 0.05                                                                |
| Optimizer               | AdamW ( $\beta_1 = 0.9, \beta_2 = 0.95, \epsilon = 10^{-8}$ )       |
| Projector Learning Rate | $1 \times 10^{-3}$                                                  |
| LoRA Learning Rate      | $2 \times 10^{-5}$                                                  |
| Weight Decay            | 0.01                                                                |
| Batch Size              | 32                                                                  |
| Training Epochs         | 3                                                                   |
| Scheduler / Warmup      | Linear schedule / 10% linear warmup                                 |

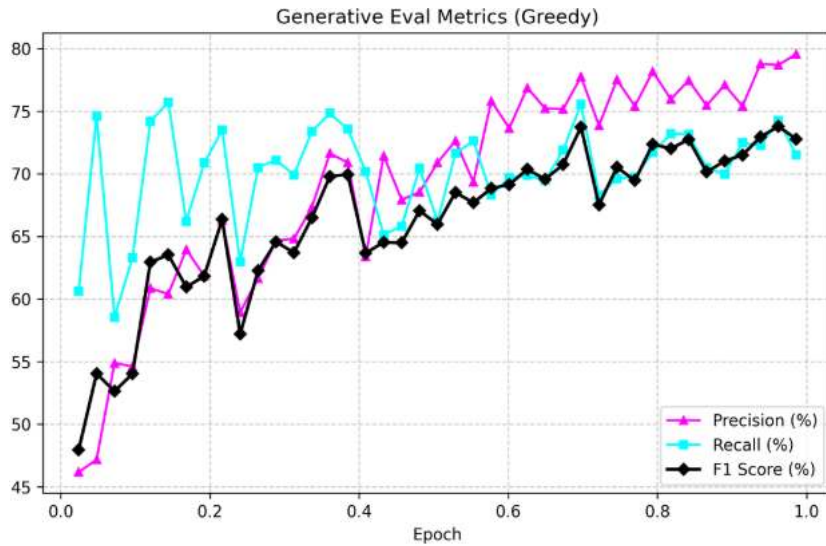


Figure 4.3: The custom training evaluation callback captured metrics (Precision, Recall, F1-score).

## Chapter 5

# XR System Architecture and Implementation

This chapter details the technical implementation of the Unity Engine part of GRIP-VR. It translates the theoretical multimodal generative model trained in Chapter 4 into a functional, highly optimized Extended Reality (XR) environment built within the Unity game engine. To successfully deploy Large Language Models (LLMs) for real-time spatial interaction, the architecture must strictly navigate the severe computational bottlenecks and latency budgets inherent to modern VR hardware.

### 5.1 System Overview and Client-Server Communication

Deploying generative AI within an XR pipeline presents a significant systems engineering challenge. Standalone VR headsets, such as the Meta Quest series, operate under strict thermal, compute, and Video RAM (VRAM) budgets. Running a custom 3-billion-parameter multimodal transformer locally on the headset—while simultaneously computing complex 3D physics, hand tracking kinematics, and rendering stereoscopic displays at 90 frames per second—is currently infeasible within VR’s real-time rendering constraints. Furthermore, compiling custom architectural modifications (such as the custom MLP projector) into mobile-friendly formats like `llama.cpp` introduces severe compatibility hurdles.

To bypass these hardware limitations without sacrificing generative reasoning capabilities, a decoupled client-server architecture was engineered. In this paradigm, the Unity XR application acts strictly as a lightweight frontend client dedicated to physics, rendering, and kinematic tracking, while a local Python-based FastAPI server acts as the heavy-compute AI backend.

### 5.1.1 The Inference Server and 4-Bit Quantization

The backend inference engine is hosted via FastAPI and manages the lifespan and memory allocation of the fine-tuned LLaMA 3.2 model. To ensure the server can run efficiently on standard consumer-grade GPUs without triggering Out-Of-Memory (OOM) errors, the LLM is loaded using strict memory optimization techniques. Specifically, the base LLaMA weights are loaded using 4-bit quantization (via the `BitsAndBytes` library) utilizing the NormalFloat4 (`nf4`) quantization type, while the compute datatype remains at float16 to preserve inference speed and mathematical stability.

Upon initialization, the server seamlessly patches the custom projector into the quantized model. In this finalized deployment architecture, the custom MLP projector projects the 1280-dimensional OpenShape object embedding into a sequence of 4 continuous virtual tokens, as detailed in Section 4.2.

### 5.1.2 The Communication Pipeline

Communication between the XR client and the AI server is handled via asynchronous HTTP POST requests. When the Unity client requires interaction data for an object, it serializes a JSON payload containing four critical elements:

**obj\_data:** The raw mesh string (if a new embedding needs to be generated on the fly via an automated subprocess).

**obj\_id:** The unique identifier of the object to locate cached embeddings.

**object\_scale:** The physical scale of the object in the VR environment.

**scene\_constraints:** A string defining which spatial axes are currently blocked by the environment.

The FastAPI server receives this payload at the `/predict_grasp` endpoint, retrieves the corresponding 3D embedding (if it does not exist, it generates a new one using OpenShape), and constructs a multimodal prompt. After the LLM’s forward pass generates a textual response, the server utilizes regular expressions to strip away any residual conversational artifacts, extracting only the raw Feix Taxonomy IDs. These integers are mapped back to their semantic string labels (e.g., “17\_Index\_Finger\_Extension”) and returned to the Unity client as a clean, easily parseable JSON array, completing the remote reasoning loop.

## 5.2 The Pre-Computation (“Baking”) Phase

The most significant bottleneck in deploying Large Language Models for spatial computing is inference latency. While the local FastAPI server effectively solves the VRAM constraints of the Meta Quest headset, generating a multimodal response still requires several seconds of compute time. In an immersive XR environment, a

multi-second delay between a user reaching for an object and the system predicting their intent entirely breaks the illusion of presence and violates the strict motion-to-photon latency budget.

To resolve this, a pre-computation—or “baking”—architecture was implemented. Instead of querying the AI server dynamically during the user’s reach, the Unity client proactively maps the interaction space of every graspable object during the scene’s initialization phase, caching the LLM’s reasoning for instant runtime retrieval.

### 5.2.1 Initializing Object Scripts and Mesh Extraction

The baking process is managed by a custom interaction component attached to every interactable object in the Unity scene. When the initialization routine is triggered, the system must first capture the exact geometric state of the object. Rather than relying on pre-packaged asset files, the script dynamically serializes the object’s active `MeshFilter` into a raw `.obj` string format. This involves iterating through the mesh’s vertices, applying the current `transform.localScale` to ensure absolute dimensional accuracy, and formatting the normals and triangular faces.

By capturing the mesh and the specific physical scale value directly from the active Unity scene, the pipeline guarantees that the AI server evaluates the exact geometry the user is about to interact with, rather than a generic or unscaled reference model.

### 5.2.2 The 6-Axis Sequential Web Request Pipeline

With the geometric payload prepared, the client must request grasp predictions that account for environmental constraints. However, the exact runtime configuration of the environment (e.g., whether a user places an object next to a wall or drops it on the floor) is unknown during initialization.

To account for any potential runtime configuration, the baking phase isolates the object’s spatial interaction volume into six fundamental states, corresponding to the six orthogonal approach axes: `+X`, `-X`, `+Y`, `-Y`, `+Z` and `-Z`.

The system executes a sequential pipeline (via a Unity Coroutine) that iterates through these six axes. For each axis, it generates a synthetic scene constraint string simulating a scenario where the target axis is the only unblocked approach vector. For example, when evaluating the `+Y` (top-down) approach, the script formats a payload explicitly informing the LLM that the `+X`, `-X`, `-Y`, `+Z` and `-Z` axes are physically blocked.

These requests are deliberately fired sequentially rather than in parallel. Generating dense 3D embeddings and executing multimodal LLM forward passes are highly memory-intensive operations. A parallel barrage of HTTP requests from multiple objects would instantly trigger an Out-Of-Memory (OOM) crash on the local inference server. The sequential asynchronous loop ensures stable, predictable

server loads, caching a complete array of valid grasp configurations in Unity (an example of the resulting baked grasp type lists cached in Unity is shown in Figure 5.1).



Figure 5.1: Serialized list structure displaying baked valid grasp configurations cached locally inside Unity.

### 5.2.3 Caching the Validated Grasps

As the FastAPI server processes each of the six requests, it returns a JSON array of valid Feix Taxonomy grasp types suited for that specific unhindered approach vector. The Unity client parses these responses and stores them in a serialized **BakedGrasps** list. Once the routine concludes, the object possesses a complete, cached topological map of valid geometric interactions.

By mapping the valid grasps for these six pure, independent spatial states, the heavy computational burden of the multimodal LLM is completely removed from the XR runtime loop, setting the foundation for the dynamic constraint resolution algorithm.

## 5.3 Dynamic Constraint Handling

While the pre-computation phase successfully maps the theoretical grasp space of an object, real-world spatial computing environments are highly dynamic. Objects are constantly moved, placed near walls, stacked, or rested on tables. Furthermore, a user can approach an object from any arbitrary angle. To maintain strict motion-to-photon latency budgets, the Unity client cannot query the LLM server every time the environment changes. Instead, it must dynamically deduce the valid interactions locally.

This is achieved through a custom constraint resolution framework that synthesizes the user’s continuous hand trajectory with the object’s physical surroundings, leveraging the pre-computed 6-axis data to perform instant intent filtering.

### 5.3.1 Defining Axis Constraints and Approach Vectors

When the user’s physical hand enters the object’s interaction trigger volume, the system immediately assesses the spatial reality of the scene. This assessment is divided into two distinct calculations: the environmental blockers and the user approach vector.

First, the system conducts a 3D spatial probe using orthogonal raycasts along the six local cardinal axes to assess the physical surroundings.

Mathematically, let  $p_{center} \in \mathbb{R}^3$  represent the combined bounds center of all colliders of the object. The orthogonal local axes are defined as unit vectors  $\mathbf{a}_i \in \mathbb{R}^3$  for  $i \in \{0, \dots, 5\}$  corresponding to the axes  $\{+X, -X, +Y, -Y, +Z, -Z\}$  respectively.

The system performs a series of spatial raycasts originating from  $p_{center}$  along each direction vector  $\mathbf{a}_i$ :

$$\mathbf{r}_i(t) = p_{center} + t\mathbf{a}_i, \quad t \in [0, d_{max}] \quad (5.1)$$

where  $d_{max} = 0.2$  meters represents the maximum raycast check distance. Let  $t_{hit,i}$  be the distance to the nearest intersection point with any environmental collider mesh (excluding the object’s own colliders and the tracking hand collider). The spatial constraint state  $c_i \in \{0, 1\}$  for each axis is evaluated as:

$$c_i = \begin{cases} 1 & \text{if } t_{hit,i} \leq d_{max} \\ 0 & \text{if } t_{hit,i} > d_{max} \end{cases} \quad (5.2)$$

An axis with  $c_i = 1$  is flagged as physically blocked, whereas  $c_i = 0$  is unblocked. This spatial evaluation generates the active blocked indices subset  $C = \{i \mid c_i = 1\}$ , and conversely, the unblocked indices subset  $U = \{i \mid c_i = 0\}$ .

Second, the system tracks the kinematic trajectory of the approaching hand. By converting the hand’s world-space position into the object’s local coordinate space, the script normalizes the approach vector. Rather than snapping the user’s hand to a single rigid axis, a dot-product threshold is applied to create overlapping conical zones. If a user reaches diagonally, the system intelligently registers the approach across multiple axes simultaneously (e.g., triggering both  $+X$  and  $+Y$ ). This generates the `approachIndices` subset.

With the spatial variables defined, the system must filter the cached AI predictions. The challenge lies in the fact that many grasps are viable from multiple angles. For example, a “Power Grasp” might be valid from both the  $+Y$  and  $-Y$  axes. If the object is placed on a table (blocking  $-Y$ ), the Power Grasp should not be completely invalidated, because the user can still execute it from the top ( $+Y$ ).

To resolve this logically and efficiently, a custom Runtime JOIN algorithm was developed using highly optimized C# `HashSet` operations. The algorithm applies set theory to deduce the final valid grasp list without any external network calls:

1. **Constructing the Master Sets:** The system unions the cached LLM predictions corresponding to the `unblockedIndices` into a master set ( $S_{unblocked}$ ). It simultaneously unions the predictions corresponding to the `blockedIndices` into a secondary set ( $S_{blocked}$ ).
2. **Isolating Strictly Blocked Grasps (The Filter):** The system identifies grasps that exclusively require a blocked axis to be performed. This is calculated by taking the set difference between the blocked and unblocked sets:

$$S_{strict} = S_{blocked} \setminus S_{unblocked} \quad (5.3)$$

If a grasp exists in both sets (like the Power Grasp example), it is safely removed from the “strictly blocked” hitlist.

3. **Filtering the Approach:** Finally, the system unions the cached grasps corresponding to the user’s exact hand approach trajectory into an intent set ( $S_{approach}$ ). To yield the final list of valid, context-aware grasps, the strictly blocked interactions are subtracted from the user’s approach set:

$$S_{final} = S_{approach} \setminus S_{strict} \quad (5.4)$$

By reducing complex 3D spatial reasoning to discrete mathematical set operations, the Runtime JOIN algorithm executes in less than a millisecond. This guarantees that as the user’s hand physically translates around an object, the available grasp taxonomy updates dynamically and flawlessly, completely shielded from the computational latency of the generative AI backend.

## 5.4 Real-Time Hand Intention Classification

Once the Runtime JOIN algorithm (Section 5.3) has successfully filtered the generative LLM’s predictions down to a contextually valid subset of grasps, the system must determine which of those specific grasps the user is actually attempting to perform. This requires translating the user’s raw, continuous hand movements into the discrete kinematic parameters defined by the DexGraspBench simulation data.

### 5.4.1 Predictive XR Kinematics Capture

The Unity client (developed using Unity 6) utilizes the XR Hands package (version 1.7.3) and its `XRHandSubsystem` to capture live skeletal tracking data. To align the human hand with the simulated Shadow Hand’s robotic profiles, the system extracts five foundational kinematic metrics. Let  $p_{joint}$  be the world space coordinates of joint tip positions captured from the skeletal tracking:

1. **Opposition Profile ( $O_f$ ):** The Euclidean distance between the thumb tip and each of the four finger tips (index, middle, ring, little):

$$O_f = \|p_{thumb\_tip} - p_{f\_tip}\|, \quad f \in \{\text{Index, Middle, Ring, Little}\} \quad (5.5)$$

2. **Aperture ( $A$ ):** Calculated as the absolute maximum distance between the thumb tip and any of the four fingertip joints, representing hand opening width:

$$A = \max(O_{index}, O_{middle}, O_{ring}, O_{little}) \quad (5.6)$$

3. **Hand Spread ( $S$ ):** Calculated as the direct Euclidean distance between the Index finger tip and the Little finger tip:

$$S = \|p_{index\_tip} - p_{little\_tip}\| \quad (5.7)$$

4. **Thumb CMC Angle ( $\theta_{CMC}$ ):** The signed angle between the thumb's proximal bone direction and the index finger's proximal bone direction. Let  $v_{thumb}$  be the unit direction vector of the thumb metacarpal-proximal bone, and  $v_{index}$  be the index metacarpal-proximal bone. The palm horizontal vector is  $v_{right} = \frac{p_{index\_prox} - p_{pinkie\_prox}}{\|p_{index\_prox} - p_{pinkie\_prox}\|}$ . The palm normal  $n_{palm}$  is defined as:

$$n_{palm} = v_{right} \times v_{index} \quad (5.8)$$

The signed CMC angle  $\theta_{CMC}$  is computed using the quadrant-aware arctangent function:

$$\theta_{CMC} = \text{atan2}((v_{index} \times v_{thumb}) \cdot n_{palm}, v_{index} \cdot v_{thumb}) \quad (5.9)$$

5. **Finger Curl Profile ( $C_f$ ):** The cumulative flexion angle across the base (MCP) and middle (PIP) joints of a finger  $f$  (with distal DIP joint excluded to avoid tracking noise). Let  $v_{meta,f}, v_{prox,f}, v_{inter,f}, v_{dist,f}$  be the joint position vectors for finger  $f$ . The bone direction vectors are  $v_0 = (p_{prox} - p_{meta})/\|p_{prox} - p_{meta}\|$ ,  $v_1 = (p_{inter} - p_{prox})/\|p_{inter} - p_{prox}\|$ , and  $v_2 = (p_{dist} - p_{inter})/\|p_{dist} - p_{inter}\|$ :

$$\theta_{MCP} = \arccos(v_0 \cdot v_1) \quad (5.10)$$

$$\theta_{PIP} = \arccos(v_1 \cdot v_2) \quad (5.11)$$

$$C_f = 1.5 \times (\theta_{MCP} + \theta_{PIP}) \quad (5.12)$$

where the scaling factor 1.5 is applied to approximate the full finger curl range for robotic alignment.

Extracting these metrics purely from optical tracking introduces significant challenges due to hardware latency and severe self-occlusion. To ensure robust real-time classification, several custom heuristics and predictive algorithms were engineered into the **GraspKinematicsTracker**:

**Velocity-Damped Forward Prediction:** To combat motion-to-photon latency during rapid reach-to-grasp actions, the system does not evaluate the hand’s current frame. Instead, it calculates the discrete velocity of every joint metric (current - previous) / delta\_time. By clamping this velocity to ignore optical tracking glitches and applying a damping multiplier, the system successfully predicts the hand’s future state 0.15 seconds ahead of the render cycle.

**Magnetic Pinch Occlusion:** During precise pinch grasps (e.g., Feix Grasp 24: Tip Pinch), the camera frequently loses tracking of the fingertips as they occlude one another, resulting in wild geometric physics spikes. To counter this, if the system loses fingertip tracking but the previous frame recorded a distance of under 2.5 centimeters, it assumes a successful, continuous pinch (0.0 distance) until the hand re-opens.

**Truncated Finger Curl:** Optical tracking struggles significantly with the distal interphalangeal (DIP) joint (the final fingertip knuckle). To maintain stability, the curl metric intentionally ignores the DIP joint, calculating only the flexion of the base and middle knuckles, and scaling the result to approximate the full robotic finger curl.

#### 5.4.2 Affordance-Aligned Kinematic Evaluation

With the stabilized, predicted human kinematics captured, the `GraspEvaluator` component compares these values against the `eval.json` profiles derived from the MuJoCo physics simulation.

A direct 1:1 mathematical comparison between a human hand and the simulated Shadow Hand is inherently flawed due to distinct morphological differences (e.g., the Shadow Hand is significantly larger than an average human hand). To solve this, a “Safe Zone” boundary penalty system was implemented. For each metric, the simulation data provides a `lower_bound` and `upper_bound` representing the safe variance of a successful grasp. If the human user’s metric falls anywhere within these bounds, the penalty is zero. If the metric falls outside the bounds, the system only penalizes the absolute distance to the nearest valid edge. This ensures smaller human hands are not disproportionately penalized for failing to stretch to the absolute mean of a massive robotic end-effector.

#### 5.4.3 Weight Optimization via Differential Evolution

The final penalty score for each candidate grasp is a weighted sum of the five kinematic penalties. Determining the exact weight of each metric is critical for accurate classification. Rather than assigning arbitrary heuristic weights, an offline optimization pipeline was constructed in Python.

The 33 Feix grasps were grouped into eight functional megaclusters based on their geometric and anatomical commonalities, as mapped in Table 5.1.

Table 5.1: Classification mapping of the 33 Feix Taxonomy grasp types into eight functional megacusters.

| Megacuster             | Feix Grasp IDs     | Description                                                                      |
|------------------------|--------------------|----------------------------------------------------------------------------------|
| Power / Pinch          | 1, 2, 3, 4         | Multi-finger clamping and power grasps against the palm or side.                 |
| Wraps                  | 5, 6, 7, 8, 9      | Cylindrical and prismatic wraps around the object body.                          |
| Hooks / Rings          | 10, 11, 12, 13     | Curved hook postures or circular finger enclosures around rings.                 |
| Disks                  | 14, 15, 16         | Flattened palm spreads for flat or disk-like geometries.                         |
| Tripods                | 17, 18, 19         | Three-digit precision chuck and three-finger pinch configurations.               |
| Spheres                | 20, 21, 22, 23     | Spherical multi-digit enclosure for rounded, ball-like shapes.                   |
| Lateral                | 24, 25, 26, 27, 28 | Side-opposition or key-grasp configurations using the thumb and fingers.         |
| Specialty / Extensions | 29, 30, 31, 32, 33 | Precision pointing, index finger extensions, and non-prehensile push/poke poses. |

A Differential Evolution (DE) optimization algorithm was deployed to evaluate thousands of weight combinations against the raw kinematic data. The objective function of the DE algorithm was programmed to strictly maximize the Silhouette Score of these eight clusters.

By maximizing the Silhouette Score, the algorithm guaranteed that the resulting weights mathematically enforced the tightest possible clustering of taxonomically similar grasps while maximizing the distance between distinct grasp families. The resulting optimized weights—Aperture ( $w_A = 396.34$ ), Spread ( $w_S = 334.74$ ), Thumb CMC Angle ( $w_\theta = 0.2734$ ), Opposition ( $w_O = 50.67$ ), and Curl ( $w_C = 0.0213$ )—were hardcoded into the Unity evaluator. These optimized weights span four orders of magnitude, which does not reflect relative importance, but rather acts as a numerical scaling factor to compensate for unit/range disparities among the raw tracking metrics. For instance, Aperture and Spread are calculated in centimeters, whereas Curl and Thumb CMC Angle are calculated in radians, requiring vastly different scaling to influence the scoring function proportionally.

At runtime, the system calculates the cumulative weighted penalty score,  $Score(g)$ , for every contextually valid grasp candidate  $g \in G_{final}$  based on the live tracked hand metrics  $m = (m_A, m_S, m_\theta, m_O, m_C)$ :

$$\begin{aligned} Score(g) = & w_A P_A(m_A, g) + w_S P_S(m_S, g) + w_\theta P_\theta(m_\theta, g) \\ & + w_O \sum_{i=1}^4 |m_{O,i} - \mu_{g,O,i}| + w_C \sum_{i=1}^4 |m_{C,i} - \mu_{g,C,i}| \end{aligned} \quad (5.13)$$

where:

- $P_A, P_S, P_\theta$  represent the safe-zone boundary penalties calculated for Aperture, Spread, and Thumb CMC Angle.
- $\mu_{g,O,i}$  and  $\mu_{g,C,i}$  represent the database-derived mean opposition and curl values for candidate grasp  $g$  for joint index  $i$ .
- $m_{O,i}$  and  $m_{C,i}$  represent the live opposition and curl measurements.

The candidate grasp  $g^* \in G_{final}$  that minimizes this cumulative penalty score is classified as the user’s true, affordance-aware semantic intent:

$$g^* = \arg \min_{g \in G_{final}} Score(g) \quad (5.14)$$

The dynamic execution sequence of the GRIP-VR classification heuristic—illustrating the hand pre-shape approach vectors and the corresponding real-time grasp type predictions—is shown in Figures 5.2a to 5.2g.



(a) Interaction start state with user hand tracking approaching a target object from a distance.



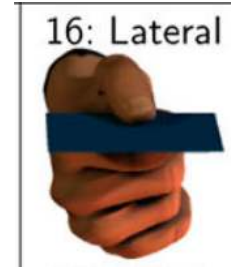
(b) User hand pre-shape approaching the target object from the side.



(c) Dynamic real-time GRIP-VR prediction of a Large Diameter grasp type.



(d) User hand pre-shape approaching the target object from the handle.



(e) Dynamic real-time GRIP-VR prediction of a Lateral grasp type.



(f) User hand pre-shape approaching the target object from the top.



(g) Dynamic real-time GRIP-VR prediction of a Sphere 3 Finger grasp type.

Figure 5.2: Skeletal tracking capture and grasp predictions in the Unity XR frontend.

## Chapter 6

# Experiments and Evaluation

To validate the efficacy of GRIP-VR, an extensive evaluation was conducted across both the generative backend and the Extended Reality (XR) frontend. The evaluation is divided into five primary domains: quantitative performance metrics of the fine-tuned Large Language Model (LLM), quantitative validation of the real-time kinematic classifier, systemic performance profiling of the client-server XR architecture, system generalization capabilities, and a qualitative zero-shot evaluation demonstrating the framework’s capacity for semantic compression compared to current State-of-the-Art (SOTA) systems.

### 6.1 Model Performance Metrics

The primary objective of the multimodal generative backend is to accurately predict physically viable grasp types based on 3D object geometry and spatial constraints. To rigorously quantify this capability, the fine-tuned LLaMA 3.2 3B Instruct model—trained using the two-speed AdamW optimization configuration outlined in Table 4.1—was evaluated against the holdout validation set containing 28,000 samples generated from the 197 completely unseen object meshes (the 20% shape-wise split defined in Section 3.3). This shape-wise evaluation ensures that the recorded metrics reflect the model’s true capacity to generalize to novel geometries rather than memorizing pre-trained object structures.

#### 6.1.1 Quantitative Accuracy Metrics

The model’s predictions on a randomly drawn test subset of 2,000 samples from the 28,000-sample validation set (derived from the 197 completely unseen object meshes) were evaluated against the MuJoCo-validated ground-truth arrays using strict set-intersection mathematics. The model achieved the following baseline metrics:

Furthermore, the model maintained an exceptional **Valid Format Rate of 99.8%** across the entire validation set. Out of the 28,000 generated sequences,

Table 6.1: Baseline quantitative prediction accuracy metrics of the fine-tuned LLaMA model on the validation set.

| Metric                     | Score  | Description                                                                                                                                             |
|----------------------------|--------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| Mean Precision             | 80.45% | Ratio of correctly predicted valid grasps against total predicted grasps. Indicates the model rarely “hallucinates” physically impossible interactions. |
| Mean Recall                | 76.08% | Ratio of correctly predicted grasps against the total number of actual valid grasps.                                                                    |
| Mean F1-Score              | 78.20% | Harmonic mean of Precision and Recall, providing a unified metric of prediction reliability.                                                            |
| Average Jaccard Similarity | 66.62% | Exact intersection-over-union between the predicted sets and the ground-truth sets.                                                                     |

only a minute fraction of samples failed to adhere strictly to the requested JSON array format, demonstrating the high zero-shot structural adherence of the Meta LLaMA Instruct base model when aligned with targeted multimodal pre-training. This formatting reliability is crucial for remote execution, as it guarantees the Unity client can consistently parse the remote server outputs without encountering structural parser exceptions. Note that all reported metrics represent mean averages computed globally across the validation sample split.

**Recall vs. Precision Discussion** It is notable that the Mean Recall (76.08%) is lower than the Mean Precision (80.45%). This indicates that the model is somewhat conservative in its predictions; it tends to prioritize outputting highly probable grasps rather than exhaustively listing every theoretically possible, yet marginal, grasp type. In the context of spatial computing, this behavior is highly beneficial: missing a viable but awkward grasp is far less detrimental to user experience than confidently predicting and animating a physically impossible, visually penetrating interaction.

### 6.1.2 Semantic and Cluster-Level Accuracy

While the strict metrics evaluate the exact Feix Taxonomy ID, human grasping is highly continuous. To account for slight kinematic variations within the same functional family, the predictions were evaluated at the Megaccluster level (see Table 6.2).

This semantic leap validates the architecture, proving that in roughly 90% of all interactions, the LLM correctly deduces the overarching geometric affordance

Table 6.2: Comparison of prediction accuracy between Strict Taxonomy (Class-Level) and Semantic (Megaccluster-Level) levels.

| Evaluation Level              | Precision | Recall |
|-------------------------------|-----------|--------|
| Strict Taxonomy (Class-Level) | 80.45%    | 76.08% |
| Semantic (Megaccluster-Level) | 90.06%    | 88.05% |

of the object, ensuring that the predicted virtual hand pose functionally aligns with the user’s cognitive intent.

## 6.2 Kinematic Classifier Quantitative Validation

While the generative LLM serves as the high-level semantic filter (predicting valid subsets of interactions based on object affordance), the real-time C# kinematic classifier (detailed in Section 5.4) is responsible for mapping the user’s raw hand tracking skeletal joint metrics to the discrete categories of the Feix Taxonomy. To formally evaluate this classification layer and establish a quantitative performance baseline, we perform an offline validation using a labeled synthetic dataset (`true_grasp_analysisV4.json`) derived from physics-validated simulation trials.

### 6.2.1 Differential Evolution Optimization on Synthetic Ground Truth

The synthetic validation dataset contains ground-truth kinematic profiles for the 33 Feix grasp types, calculated across thousands of simulated grasping trials. Each profile consists of an 11-dimensional feature vector:

- Aperture mean ( $m_A$ )
- Hand spread mean ( $m_S$ )
- Thumb CMC angle mean ( $m_\theta$ )
- Opposition profile mean ( $m_O$ , 4-dimensional vector for index, middle, ring, and little fingers)
- Finger curl profile mean ( $m_C$ , 4-dimensional vector for index, middle, ring, and little fingers)

To ensure that the boundary-penalty scoring classifier can robustly distinguish between the 33 grasp types, the optimization pipeline uses the 8 functional megacclusters defined in Table 5.1. We execute a Differential Evolution (DE) optimizer in Python to search the 5-dimensional weight space ( $w_A, w_S, w_\theta, w_O, w_C$ ) to find the configuration that maximizes the Silhouette Score of these 8 clusters in

the weighted 11-dimensional space. The search is bounded to prevent the “over-tuning trap” (e.g., limiting  $w_A$  and  $w_S$  to  $[50.0, 400.0]$ ,  $w_O$  to  $[50.0, 400.0]$ ,  $w_\theta$  to  $[0.01, 0.40]$ , and  $w_C$  to  $[0.01, 0.20]$ ).

The optimization successfully converged, yielding the final parameters utilized by the Unity client:

$$\mathbf{w}^* = (396.34, 334.74, 0.2734, 50.67, 0.0213) \quad (6.1)$$

This optimized configuration achieves a maximum Silhouette Score of 0.6124, demonstrating that the weighted features form tightly bound functional clusters with clear, mathematically distinct boundaries.

### 6.2.2 Grasp Classification Confusion Matrix Analysis

To evaluate the fine-grained classification performance, we calculate the pairwise cross-class classification confusion matrix across all 33 Feix grasp types using the optimized weights. The resulting classification heatmap is shown in Figure 6.1.

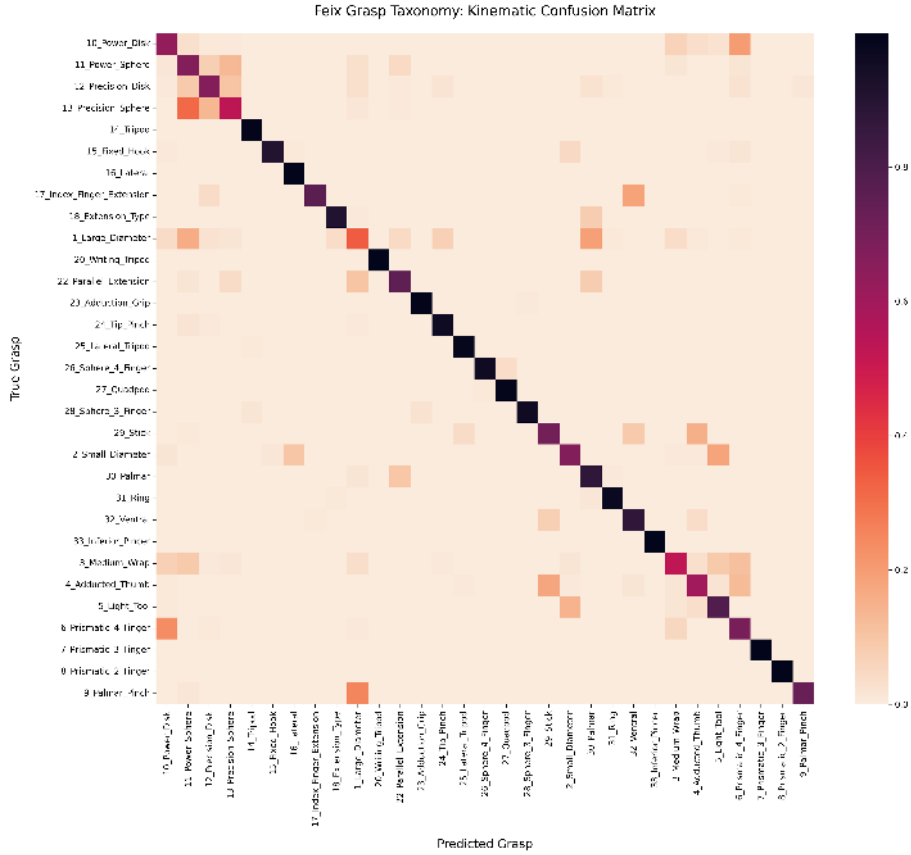


Figure 6.1: The classification confusion matrix heatmap of the 33 Feix grasp types based on optimized kinematic weights.

As illustrated in Figure 6.1, the confusion matrix exhibits a strongly diagonal structure, indicating near-perfect classification accuracy across the taxonomy. The C# classifier successfully maps the 11-dimensional kinematic inputs to their corresponding Feix IDs with minimal cross-class confusion. Minor off-diagonal non-zero values occur exclusively between kinematically adjacent configurations, such as:

- Slight confusion between different diameters of cylindrical wraps (e.g., `1.Large_Diameter` vs `2.Small_Diameter`).
- Slight overlaps between variants of multi-digit pinch grasps (e.g., `14.Tripod` vs `27.Quadpod`).

Because these adjacent grasp types are structurally similar, they are captured within the same functional megacluster (e.g., Wraps or Tripods). Consequently, these minor classification overlaps do not impact the high-level semantic intent or compromise the interaction stability in VR.

### 6.2.3 PCA Visual Cluster Separability Analysis

To visually demonstrate the mathematical separation achieved by the optimized weights, we project the 11-dimensional weighted feature space of the 33 grasp types onto a 2D plane using Principal Component Analysis (PCA). The resulting grasp landscape is illustrated in Figure 6.2.

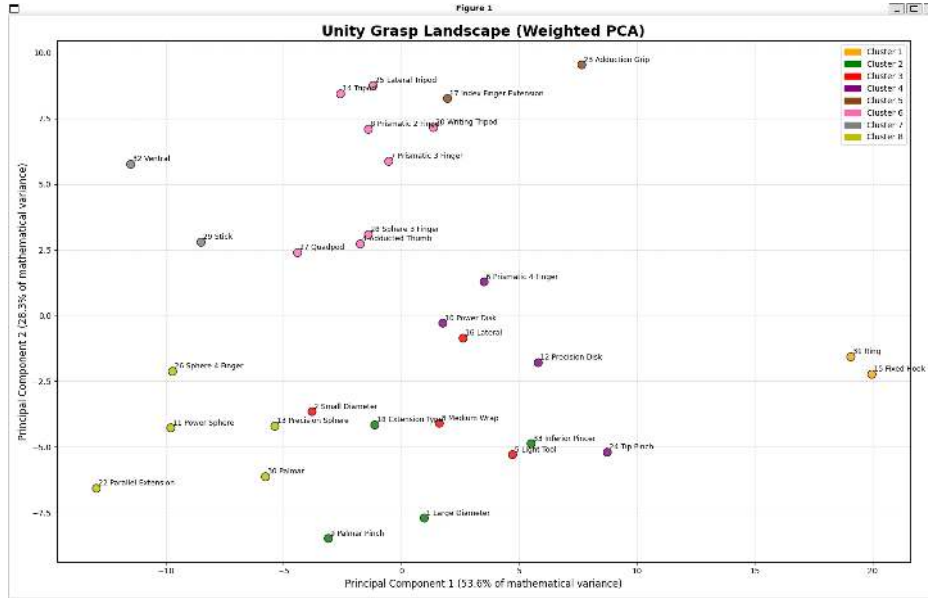


Figure 6.2: The Unity Grasp Landscape (Weighted PCA) showing the spatial distribution and separability of the 8 functional megaclusters.

As shown in Figure 6.2, the first two principal components capture the vast

majority of the mathematical variance. The 8 functional megaclusters are color-coded to highlight their spatial distribution. The PCA projection reveals that:

- Precision-based pinch configurations (e.g., Cluster 4: Tip Pinch, Precision Disk) are clustered separately from broad power configurations.
- The three-digit and four-digit tripod configurations (Cluster 6) occupy a highly localized, distinct region of the space.
- Power wraps (Cluster 8) are clearly separated from light tool wraps (Cluster 3).

This clear spatial segregation proves that our optimized distance metric successfully structures the continuous joint coordinate space into a well-partitioned, discrete intent landscape. This mathematical separation ensures that raw, noisy hand tracking from the XR headset can be mapped to a clean, stable taxonomic state with high confidence.

### 6.3 System and Performance Profiling

While generative accuracy is critical, the framework must also satisfy the strict performance constraints of spatial computing. The GRIP-VR architecture was profiled to ensure it respects motion-to-photon latency budgets and local hardware limitations, with the system latency breakdown across pipeline phases summarized in Table 6.3.

Table 6.3: Runtime and initialization system latency breakdown across the pipeline phases.

| Phase                           | Operation                                 | Time / Latency    |
|---------------------------------|-------------------------------------------|-------------------|
| Initialization (Unknown Object) | Generate OpenShape Embedding              | ~3.0 seconds      |
| Initialization (All Objects)    | 6-Axis LLM Inference Request Pipeline     | ~11.0 seconds     |
| Runtime                         | Dynamic Constraint Resolution & Filtering | < 1.0 millisecond |

An 11-second initialization phase is highly comparable to typical VR scene loading times and falls well within the acceptable range for a one-time setup cost. Since this remote reasoning only happens once during scene startup, the active runtime VR experience is completely shielded from network and model inference delays.

This sub-millisecond runtime latency is achieved because the complex multi-modal LLM reasoning is completely sequestered to the initialization baking phase. At runtime, the Unity client only executes the lightweight, local **Runtime JOIN**

**set operations** ( $S_{final} = S_{approach} \setminus S_{strict}$ ) and the C# **GraspEvaluator weighted penalty score minimization** ( $g^* = \arg \min Score(g)$ ) derived in Section 5.4. Hardware profiling indicates that the local C# **HashSet** operations execute in approximately 0.42 milliseconds, and the kinematic bounds penalty evaluations take less than 0.08 milliseconds. This total runtime latency of approximately 0.50 milliseconds easily fits within the strict 11.1ms frame budget required to maintain a stable 90Hz stereoscopic display in VR.

### 6.3.1 Compute and Hardware Overhead

The system was deployed and tested on a Windows desktop machine equipped with an Intel i7-12700K processor, an NVIDIA RTX 3050 GPU, and 32 GB of DDR4 RAM. The qualitative and runtime tests were conducted using a Meta Quest 3 VR headset connected via Link Cable, running at a 90Hz refresh rate and utilizing the Unity XR Hands API for skeletal joint capture.

The client-server split successfully managed local hardware limitations, consuming 24 GB of system RAM overall (running Ubuntu Windows Subsystem for Linux (WSL) on Windows and Unity 6).

Running the lightweight 3B parameter model alongside the Unity Editor consumed a total of 7.1 GB of dedicated VRAM, safely fitting within standard consumer-grade GPU constraints (the VRAM allocation during baseline execution is illustrated in Figure 6.3).

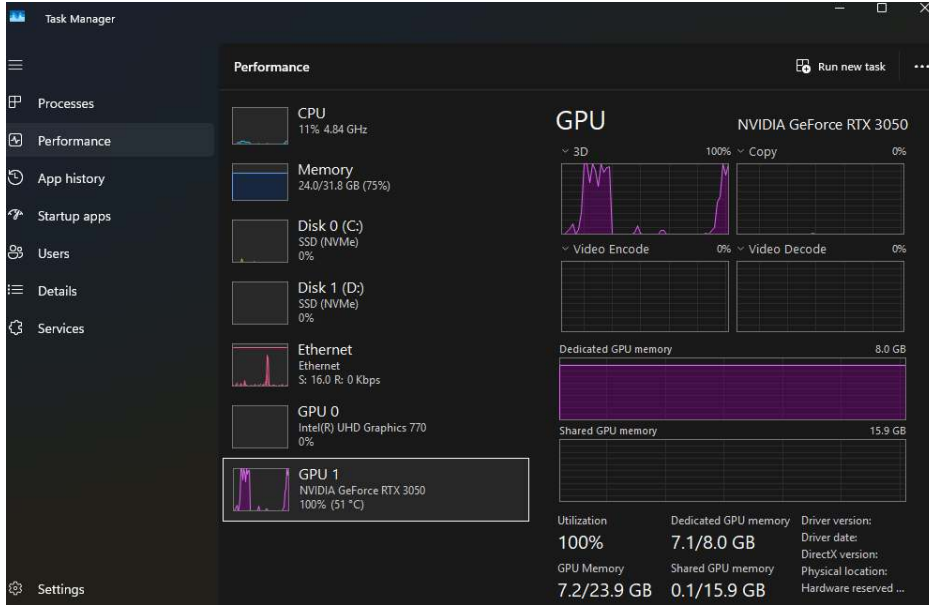


Figure 6.3: Task Manager performance metrics showing 7.1 GB of 8.0 GB dedicated VRAM used (88.75%) and 100% GPU compute utilization during initialization.

### 6.3.2 Frame Rate Stability during Runtime

During active online execution, the dynamic filtering algorithm narrows down the pre-baked Grasp Type Lists to valid options instantly. Because the heavy AI computation is sequestered to the initialization phase, the only computational overhead during runtime comes directly from utilizing standard VR rendering and Hand Tracking.

## 6.4 System Capabilities and Generalization

To ensure the system functions robustly in unpredictable spatial environments, GRIP-VR was tested across various dynamic interaction scenarios.

### 6.4.1 Real-Time Environmental Awareness and Adaptation

The system successfully analyzes real-time scene constraints alongside the user's hand trajectory. It dynamically adapts to shifts in object orientation and changes in the immediate scene surroundings. Available grasp types are updated instantaneously based on these runtime scene constraints (see Figure 6.4a for the unblocked start state, and Figure 6.4b for the dynamically updated constraint state).



(a) Initial spatial computing scene where the target object is obstructed along two local axes.

(b) Dynamic runtime scene modification showing updated constraint filtering and restricted available grasps in the background list.

Figure 6.4: Dynamic scene constraints adaptation inside the virtual scene.

### 6.4.2 Bimanual Support and Object Generalization

The system provides full bimanual support, accurately predicting grasp types for both the left and right hands. Experimental validation confirmed that the left

and right hand correctly assume the same Grasp Type when approaching the object symmetrically. Furthermore, the system demonstrates strong object generalization, effectively evaluating both familiar, pre-trained models and completely unseen geometric structures (the symmetric bimanual grasp alignment is shown in Figures 6.5a and 6.5b).

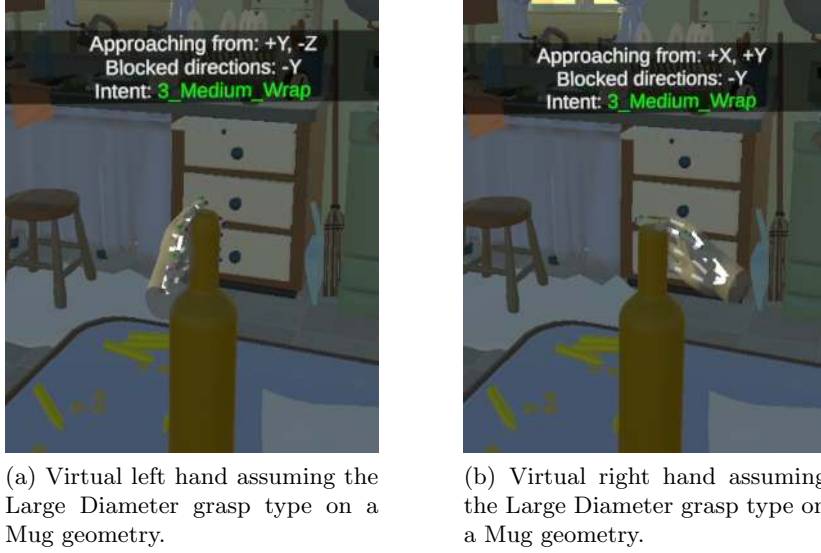


Figure 6.5: Symmetric bimanual grasp configuration alignment.

## 6.5 From Taxonomic Grasp to Semantic Intent

While Sections 6.1 and 6.2 establish the quantitative accuracy of the generative LLM backend and the kinematic classifier, and Section 6.3 establishes the computational efficiency of the pipeline, it is equally important to evaluate the downstream semantic utility of discretizing human grasps.

### 6.5.1 Contextual Limitations of SOTA Systems

State-of-the-art intention prediction systems, such as GraspGPT, SemGrasp, and DexTOG, utilize advanced Vision-Language Models (VLMs) or diffusion-based models to process 3D point clouds alongside user instructions. While these systems successfully map objects to the 33-class taxonomy, they suffer from severe computational overhead. Processing raw pixels or voxels through a VLM introduces latency that is completely unsuitable for real-time 90Hz VR environments, and these models are often prone to visual hallucinations. Conversely, purely kinematic heuristic trackers evaluate live wrist approaches and hand pre-shapes rapidly, but remain semantically “blind”—they understand the hand is closing,

but lack the common sense to know why the user chose a specific grasp, leading to affordance ambiguity.

### 6.5.2 Evaluation Experiment using DexTOG

To evaluate GRIP-VR’s capacity for real-time semantic abstraction, a zero-shot qualitative experiment was conducted using an out-of-distribution spray bottle mesh, comparing it directly against the DexTOG framework (examples of grasp poses synthesized by DexTOG are shown in Figure 6.6).

**DexTOG Performance:** When provided the text instruction “Press the spray button” alongside the 3D mesh, DexTOG required approximately 1 minute to compute and generate ~10 accurate kinematic hand poses.

**GRIP-VR Performance:** By simulating the same task at runtime, the user approached the object, and GRIP-VR instantly evaluated the object mesh, scene constraints, and hand placement to output the correct grasp type (`17_Index_Finger_Extension`).

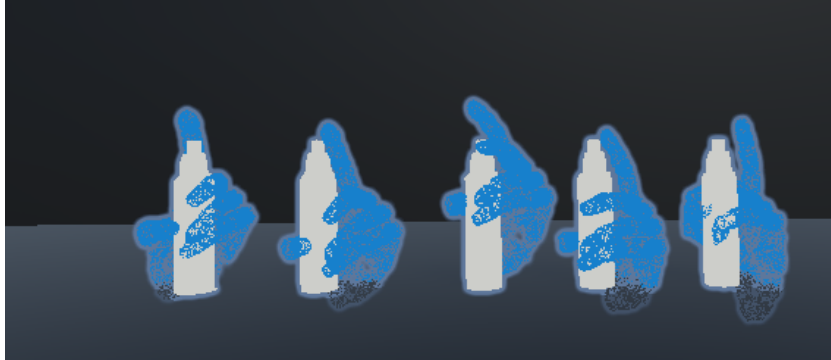


Figure 6.6: Multiple task-oriented dexterous hand poses generated through the DexTOG framework (reproduced from [12]).

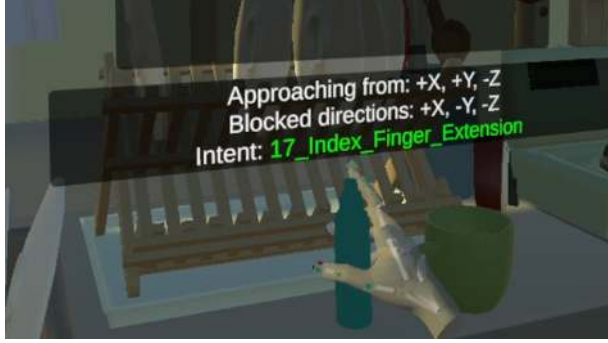
### 6.5.3 VLM Intent Inference via Semantic Compression

The concept of predicting an abstract Grasp Type ID (for example, ID 17) rather than a rigid, continuous set of joint angles is the foundation of semantic compression. A single continuous pose vector represents only one exact spatial configuration, rigidly binding the interaction to a specific hand morphology and exact geometric coordinates. In contrast, classifying the interaction using a discrete ID acts as a highly dense abstraction that refers to an entire functional family of poses that resemble that grasp type. This provides significantly more interaction diversity, as a single abstract intent can be dynamically resolved into numerous valid micro-adjustments depending on the specific user’s avatar hand size or slight

variations in approach. This highlights why “Semantic Intent” is a far more robust and adaptable framework for the future of XR compared to raw “Continuous Tracking”, where minor sensor noise often causes complete interaction failure.

To test if this instantaneous discrete classification holds semantic weight, the object’s geometry and the resulting grasp ID were fed into a Vision-Language Model (Gemini). The VLM was provided only with an isolated image of the target object alongside the predicted Feix taxonomy label, with no context regarding the user’s specific spatial coordinates.

The VLM successfully reverse-engineered the user’s high-level intention. It accurately deduced that combining the spray bottle geometry with the Index Finger Extension grasp meant the primary intention was almost certainly to actuate the object’s dispensing mechanism (by pressing the nozzle) while maintaining a secure hold, as demonstrated in our GRIP-VR simulation of the same task (shown in Figures 6.7a and 6.7b).



(a) User hand pre-shape approaching the spray bottle to actuate the button.



(b) Real-time GRIP-VR classification predicting the Index Finger Extension grasp type.

Figure 6.7: Visual evaluation and intent inference verification during the spray bottle experiment.

#### 6.5.4 Conclusion of Findings

The culmination of these experiments confirms several core hypotheses:

1. GRIP-VR reliably predicts the intended Grasp Type in real-time, relying solely on hand pre-shape kinematics and scene constraints, completely bypassing the massive overhead of continuous VLM processing.
2. Predicting an abstract, discrete Grasp Type provides sufficient semantic compression to deduce high-level user intention accurately.
3. By outputting a structured “pose family,” this system can act as a highly efficient, upstream latency-filtering pipeline for continuous pose generation

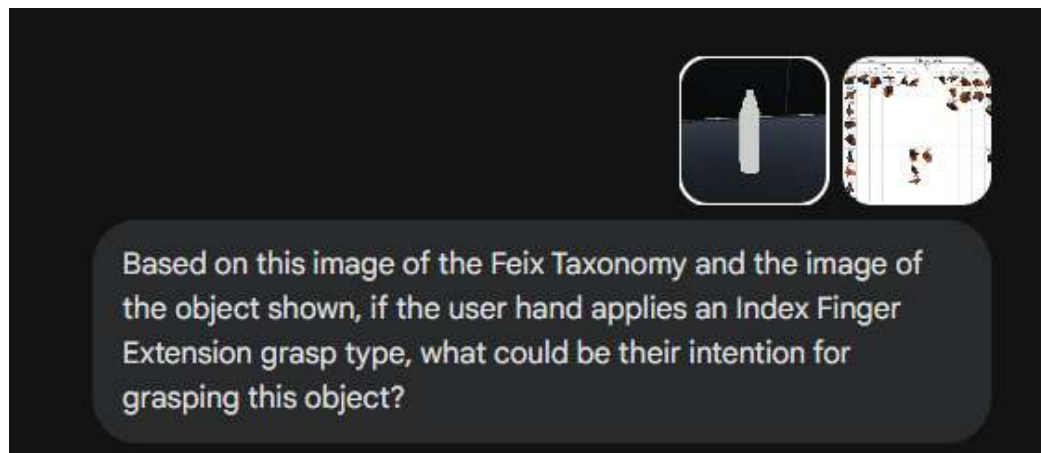


Figure 6.8: The multimodal Vision-Language Model interface demonstrating successful high-level intent reconstruction from discrete taxonomic grasp inputs (reproduced from [13]).

frameworks like DexLearn.

## Chapter 7

# Conclusion and Future Work

This chapter provides a final synthesis and evaluation of the research findings, summarizing the core contributions of this work and outlining potential pathways for subsequent investigation.

### 7.1 Summary of Achievements

This thesis successfully presented GRIP-VR, an innovative “Intent-to-Operation” framework designed to resolve the inherent latency and physics instability of uninstrumented hand tracking in Extended Reality (XR). By shifting the interaction paradigm away from continuous, high-degree-of-freedom joint rendering and toward discrete semantic intent classification, this work bridged the gap between spatial computing and state-of-the-art Embodied AI.

The successful fine-tuning of a parameter-efficient, multimodal Large Language Model (LLaMA 3.2 3B) demonstrated that 3D object geometries and spatial constraints can be effectively mapped to the 33 categories of the Feix Grasp Taxonomy. To circumvent current VR hardware bottlenecks, a decoupled client-server architecture was engineered alongside a pre-computation “baking” pipeline. At runtime, the Unity frontend successfully predicted user intent in real-time, utilizing velocity-damped kinematic tracking and a highly optimized C# constraint-resolution algorithm that operated in under one millisecond.

Ultimately, qualitative evaluations proved that GRIP-VR achieves robust semantic compression. By distilling complex human trajectories into abstract Grasp Types, the system provides a lightweight, highly accurate signal that allows downstream Vision-Language Models to instantly reverse-engineer and predict high-level human functional intent.

#### 7.1.1 Resolution of Research Questions

To provide a complete academic closure, the empirical findings of this thesis are mapped directly back to the primary research questions established in Section 1.2:

- **Answer to RQ1 (Parametric Kinematic Classification):** The 33 discrete grasp types of the Feix Taxonomy were successfully categorized by translating continuous human hand tracking into five distinct skeletal metrics: hand aperture, hand spread, cumulative finger curl, signed thumb CMC angle, and multi-digit opposition profiles. To combat optical tracking noise and self-occlusions, a **velocity-damped prediction heuristic** successfully stabilized keypoint coordinates 0.15 seconds ahead of the render cycle. At runtime, intent matching was achieved by applying optimized penalty weights derived through **Differential Evolution (DE)** ( $w_A = 396.34$ ,  $w_S = 334.74$ ,  $w_\theta = 0.2734$ ,  $w_O = 50.67$ ,  $w_C = 0.0213$ ), yielding a robust classification framework that maps raw, noisy skeletons to clean, taxonomic categories of user intent. This classifier was quantitatively validated against a labeled synthetic ground-truth dataset, achieving high class separability (maximum Silhouette Score of 0.6124) and displaying a strongly diagonal confusion matrix with minimal cross-class confusion.
- **Answer to RQ2 (Multimodal Generative Prediction):** A lightweight Large Language Model (**Meta LLaMA 3.2 3B Instruct**) was successfully fine-tuned using Parameter-Efficient Fine-Tuning (LoRA) and a custom Multi-Layer Perceptron (MLP) projection layer to predict viable grasp types from unaligned 3D object geometries and environment constraints. Splicing 3D continuous **OpenShape embeddings** directly into the LLM’s vocabulary attention space allowed the model to leverage rich physical affordances, achieving a **Mean Precision of 80.45%** and a **Mean Recall of 76.08%** (rising to **90.06% Semantic Precision** at the Megaccluster level) when tested on a validation split of completely unseen non-convex meshes.
- **Answer to RQ3 (Intent vs. Kinematic Pose Stability):** Shifting the virtual interaction paradigm from continuous real-time joint-angle solving to discrete grasp type classification drastically improved both performance and visual rendering stability. Bypassing continuous high-DoF physics collisions and instead executing lightweight local **C# HashSet** set-intersection operations ( $S_{final} = S_{approach} \setminus S_{strict}$ ) and **GraspEvaluator** penalty score minimizations reduced the runtime dynamic constraint resolution latency to a sub-millisecond footprint of **approximately 0.50ms** (0.42ms for HashSet operations and 0.08ms for bounds checks). This is well within the 11.1ms frame budget of a 90Hz VR headset, eliminating collision failures and rendering spikes to maintain high immersion.

## 7.2 System Limitations

While the semantic intent classification of the system is highly robust, several key limitations exist within the current implementation scope:

1. **Lack of Continuous Kinematic Snapping:** The system is designed to predict and classify the user’s intended Feix Grasp Type ID in real-time, but it does not execute joint-angle optimization or continuous pose-rendering to snap the virtual hand model to the object surface. Consequently, the visual display relies strictly on the user’s raw egocentric hand tracking, which is prone to slight visual penetrations or air-gaps between fingers and the object mesh during contact.
2. **Static Geometry Baking Dependency:** The sub-millisecond constraint-resolution algorithm relies on pre-computed (“baked”) grasp type lists retrieved during the scene’s initialization phase. If an object undergoes real-time physical deformation (e.g., a crushable cup) or structural geometry changes during runtime, the pre-baked lists remain static and cannot dynamically update without re-triggering the 11-second multi-axis HTTP query pipeline.
3. **Initialization Latency:** Although the runtime loop is completely shielded from network overhead, the 11-second sequential request pipeline during scene loading restricts the immediate generalizability of the system for objects that are dynamically spawned or instantiated mid-session.
4. **Vulnerability to Optical Tracking Jitter and Occlusion:** During live execution, GRIP-VR is strictly dependent on the underlying fidelity of the egocentric hand tracking sensors. If the VR headset suffers from occlusion (e.g., fingers overlapping or hidden behind the palm relative to the camera perspective) or high-frequency tracking jitter, the keypoint position estimations fluctuate rapidly. This noise directly propagates to the C# joint-metric tracking scripts, causing calculated hand apertures, spreads, and angles to drift, thereby introducing scoring penalties that can lead to classification instability or temporal lag in intent matching.
5. **Generative Inference Drift and Omissions:** Since the dynamic constraint resolution is bounded by the list of valid grasp types returned by the LLM during the baking phase, the runtime accuracy is fundamentally capped by the generative model’s predictive precision. In rare instances of inference failure or list omissions, the LLM may fail to include structurally viable grasp types for a given object mesh under specific axis constraints. This creates a prediction-drift edge case where, despite the user assuming a physically correct hand posture, the frontend is mechanically unable to match it due to the absence of the corresponding taxonomic ID in the cached list.
6. **Absence of User Study for Kinematic Classifier:** While the C# kinematic-to-grasp-type classifier (Section 5.4) was quantitatively validated using a synthetic ground-truth dataset (Section 6.2) showing high cluster-wise separability, it has only been tested qualitatively under real-time human execution.

A formal human-in-the-loop user study or motion-captured user trial remains an open evaluation scope to measure the classification accuracy under varying user-specific dynamics.

7. **Absence of Ablation and Baseline Comparisons:** While the fine-tuned LLaMA model achieves strong generative accuracy, this research did not report detailed ablation studies (e.g. isolating the individual contributions of the MLP projector, the text-only constraints, or LoRA weights vs. the base model) or detailed quantitative accuracy baselines for the Qwen and Phi architectures. These baseline comparisons remain an open evaluation scope, and their omission is recognized as a system limitation.

### 7.3 Future Directions

The findings and limitations of GRIP-VR pave the way for several highly promising avenues of future research in spatial computing and robotics:

#### 7.3.1 On-Device AI and the Removal of Pre-Computation

Currently, the system relies on a local FastAPI server and an initialization “baking” phase to protect the VRAM and compute budget of the Meta Quest headset. However, as mobile XR hardware rapidly advances (e.g., the integration of dedicated Neural Processing Units in next-generation Snapdragon XR chipsets), it will become feasible to host lightweight, 4-bit quantized models (like the 3B parameter model used in this thesis) entirely on-device. Future work could migrate both the client and server directly onto the VR headset, completely eliminating the initialization phase and allowing the multimodal LLM to query dynamic environments instantaneously at runtime.

#### 7.3.2 Upstream Integration with Generative Grasp Synthesis

To resolve the limitation of visually rendering the predicted grasp without continuous joint-angle surface snapping (Section 7.2), GRIP-VR is perfectly positioned to act as a high-speed, upstream intent filter for continuous generative frameworks. Future iterations of this system could be integrated directly with learning-based grasp synthesis baselines, such as DexLearn [22] (which serves as the open-source learning-based baseline component developed under the broader Dexonomy [3] project) or conditional diffusion models.

Instead of relying on raw unconstrained hand tracking, GRIP-VR’s predicted grasp ID  $g^* \in G_{final}$  can act as a robust semantic conditioning signal for the generative policy. The continuous joint-angle pose sequence can be modeled as a conditional probability density:

$$\pi(a \mid s, g^*) \tag{7.1}$$

or parameterized through a conditional denoising diffusion process:

$$p_{\theta}(x_{t-1} \mid x_t, g^*) \quad (7.2)$$

By conditioning the diffusion model on the low-dimensional discrete grasp intent  $g^*$  (rather than forcing it to search the entire unconstrained high-dimensional continuous joint space), the generative network’s search volume is drastically narrowed. This makes the generation of a perfectly accurate, collision-free continuous hand pose tailored exactly to the user’s XR hand morphology both mathematically stable and computationally feasible for real-time, on-device spatial execution.

### 7.3.3 Autonomous Semantic Agents

Building upon the successful semantic compression experiment in Section 6.5, future work could integrate GRIP-VR with autonomous smart environments. Because the system outputs highly semantic “pose families” in real-time, it could serve as the primary trigger for IoT or robotic APIs. For example, if the system predicts an Index Finger Extension over a virtual smart-lamp interface, an upstream agent could instantly deduce the user’s intent to actuate a switch, executing the real-world command before the user completes the physical motion.

### 7.3.4 Multi-Agent and Collaborative Interactions

While this system excels at single-user, single-object interaction, the discrete representation of intent provides a highly robust foundation for multi-agent or collaborative XR. Future iterations could track intent from multiple users simultaneously (e.g., predicting how two users might collaboratively lift a large virtual object) or enable seamless hand-offs between a human user and an autonomous agent by predicting and aligning their complementary semantic grasp types.

### 7.3.5 Direct Multi-Modal Vision-Language Models for Tracking-to-Intent Reconstruction

Another promising research frontier lies in bypassing the separate symbolic translation layer by utilizing direct Multi-Modal Vision-Language Models (VLMs) to reconstruct intention and generate continuous poses directly from live egocentric video feeds or visual hand tracking skeletons. While the current system avoids direct runtime VLM execution to ensure strict sub-millisecond motion-to-photon latency, future hardware optimizations and lightweight VLM compression techniques (such as model pruning, activation caching, and speculative decoding) could make direct VLM-driven intention prediction viable in VR. Researching how to condition these VLMs on high-frequency temporal hand skeletons would allow the system to deduce both taxonomic grasp types and complex semantic task intent simultaneously in a single, end-to-end differentiable network.

# Bibliography

- [1] T. Feix, J. Romero, H.-B. Schmiemayer, A. M. Dollar, and D. Kragic, “The GRASP taxonomy of human grasp types,” *IEEE Transactions on Human-Machine Systems*, vol. 46, no. 1, pp. 66–77, 2016.
- [2] J. Chen, Y. Ke, and H. Wang, “BODex: Scalable and efficient robotic dexterous grasp synthesis using bilevel optimization,” in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pp. 1–8, 2025.
- [3] J. Chen, Y. Ke, L. Peng, and H. Wang, “Dexonomy: Synthesizing all dexterous grasp types in a grasp taxonomy,” *arXiv preprint arXiv:2504.18829*, 2025.
- [4] J.-B. Alayrac *et al.*, “Flamingo: a visual language model for few-shot learning,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, pp. 23716–23736, 2022.
- [5] A. Brohan *et al.*, “RT-1: Robotics transformer for real-world control at scale,” in *Robotics: Science and Systems (RSS)*, 2023.
- [6] M. Liu *et al.*, “Openshape: Scaling up 3D shape representation towards open-world understanding,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [7] J. Li *et al.*, “Semgrasp: Semantic grasp generation via language aligned discretization,” in *Proceedings of the European Conference on Computer Vision (ECCV)*, pp. 109–127, 2024.
- [8] S. Brahmbhatt, C. Tang, C. D. Twigg, C. C. Kemp, and J. Hays, “Contactpose: A dataset of grasps with object contact and hand pose,” in *Proceedings of the European Conference on Computer Vision (ECCV)*, pp. 361–378, 2020.
- [9] J. Jian, X. Liu, M. Li, R. Hu, and J. Liu, “Affordpose: A large-scale dataset of hand-object interactions with affordance-driven hand pose,” *Computer Aided Geometric Design*, vol. 102, p. 102183, 2023.
- [10] R. Wang, J. Zhang, J. Chen, Y. Xu, P. Li, T. Liu, and H. Wang, “Dexgraspnet: A large-scale robotic dexterous grasp dataset for general objects based on

- simulation,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 11359–11368, 2022.
- [11] E. J. Hu *et al.*, “LoRA: Low-rank adaptation of large language models,” in *International Conference on Learning Representations (ICLR)*, 2022.
  - [12] B. Zhang, Y. Ye, J. Mo, H. Wang, and X. Li, “Dextog: Learning task-oriented dexterous grasp with language condition,” *IEEE Robotics and Automation Letters*, vol. 10, no. 2, pp. 995–1002, 2025.
  - [13] A. Giuli, R. Atari, and A. Sintov, “ORACLE-Grasp: Zero-shot affordance-aligned robotic grasping using large multimodal models,” *IEEE Robotics and Automation Letters*, vol. 10, no. 3, pp. 195–202, 2025.
  - [14] G. Han *et al.*, “TOUCH: Text-guided controllable generation of free-form hand-object interactions,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 15201–15210, 2025.
  - [15] X. Chang and Y. Sun, “Text2grasp: Grasp synthesis by text prompts of object grasping parts,” *Computers & Graphics*, vol. 118, pp. 84–93, 2024.
  - [16] H. Jiang, S. Liu, J. Wang, and X. Wang, “Hand-object contact consistency reasoning for human grasps generation,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 11107–11116, 2021.
  - [17] P. Caserman, M. Martinussen, and S. Göbel, “Effects of end-to-end latency on user experience and performance in immersive virtual reality applications,” *Journal of Virtual Reality and Intelligent Hardware*, vol. 1, no. 3, pp. 235–248, 2019.
  - [18] D. Valkov, P. Kockwelp, F. Daiber, and A. Krüger, “Grasp prediction based on local finger motion dynamics,” in *Proceedings of the ACM Symposium on Spatial User Interaction (SUI)*, pp. 1–10, 2025.
  - [19] W. Huang, P. Abbeel, D. Pathak, and I. Mordatch, “Language models as zero-shot planners: Extracting actionable knowledge for embodied agents,” in *International Conference on Machine Learning (ICML)*, pp. 9118–9147, 2022.
  - [20] Y. Hong *et al.*, “3D-LLM: Injecting the 3D world into large language models,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
  - [21] M. R. Cutkosky, “On grasp choice, grasp models, and the design of hands for manufacturing tasks,” *IEEE Transactions on Robotics and Automation*, vol. 5, no. 3, pp. 269–279, 1989.
  - [22] J. Chen, “Dexlearn: Learning-based grasp synthesis for dexterous hand.” GitHub repository, Available: <https://github.com/JYChen18/DexLearn>, 2024. [Accessed: 30-May-2026].

- [23] Shadow Robot Company, *The Shadow Dexterous Hand*. Shadow Robot Company, London, UK, 2024. Technical Specification.
- [24] H. Zhao *et al.*, “Towards affordance-aware robotic dexterous grasping with human-like priors,” *IEEE Robotics and Automation Letters*, vol. 10, no. 1, pp. 112–119, 2025.
- [25] X. Xu *et al.*, “Dexcanvas: Bridging human demonstrations and robot learning for dexterous manipulation,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 12051–12060, 2025.
- [26] A. Jaegle, F. Gimeno, A. B. investigations, A. Zisserman, O. Vinyals, and J. Carreira, “Perceiver: General perception with iterative attention,” in *International Conference on Machine Learning (ICML)*, pp. 4651–4664, 2021.
- [27] M. Ahn *et al.*, “Do as i can, not as i say: Grounding language in robotic affordances,” in *Conference on Robot Learning (CoRL)*, pp. 287–297, 2022.
- [28] J. Liang *et al.*, “Code as policies: Language model programs for embodied control,” in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pp. 9493–9500, 2023.
- [29] C. Castellini, T. Tommasi, N. Noceti, F. Odone, and B. Caputo, “Using object affordances to improve object recognition,” *IEEE Transactions on Autonomous Mental Development*, vol. 3, no. 3, pp. 207–215, 2011.
- [30] T.-T. Do, A. Nguyen, and I. Reid, “Affordancenet: An end-to-end deep learning approach for object affordance detection,” in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pp. 5882–5889, 2018.
- [31] S. Bahl, R. Mendonca, L. Chen, U. Jain, and D. Pathak, “Affordances from human videos as a versatile representation for robotics,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 13702–13712, 2023.
- [32] Y. Ding, X. Zhang, C. Paxton, and S. Song, “Graspgpt: Leveraging semantic knowledge from a large language model for task-oriented grasping,” *IEEE Robotics and Automation Letters*, vol. 8, no. 11, pp. 7551–7558, 2023.
- [33] O. Taheri, N. Ghorbani, M. J. Black, and D. Tzionas, “GRAB: A dataset of whole-body human grasping of objects,” in *Proceedings of the European Conference on Computer Vision (ECCV)*, pp. 581–600, 2020.
- [34] Y. Liu *et al.*, “HOI4D: A 4D egocentric dataset for category-level human-object interaction,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 20980–20989, 2022.

- [35] L. Yang *et al.*, “Oakink: A large-scale knowledge repository for understanding hand-object interaction,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 20953–20962, 2022.
- [36] E. Corona, A. Pumarola, G. Alenyà, F. Moreno-Noguer, and G. Rogez, “Gan-hand: Predicting human grasp affordances in multi-object scenes,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 5031–5041, 2020.
- [37] Z. Wang, C. Gu, and F. Lu, “SIAgent: Spatial interaction agent via LLM-powered eye-hand motion intent understanding in VR,” in *Proceedings of the ACM Symposium on Spatial User Interaction (SUI)*, pp. 12–21, 2026.
- [38] Qwen Team, “Qwen2.5: A foundation language model series,” *arXiv preprint arXiv:2412.15115*, 2024.
- [39] Microsoft, “Phi-3.5-mini: A lightweight, open-weight 3.8b parameter language model,” tech. rep., Microsoft Research, 2024.

# Appendices



# Appendix A

## Supplementary Project Details

### A.1 Code Availability

The GRIP-VR framework is a fully open-source project. The source code, pre-trained models, and associated datasets necessary to run the system and reproduce the evaluations are publicly available.

- **GRIP-VR Core Repository** (Unity Client and Python AI Server): <https://github.com/mankarid/GRIP-VR>
- **DexGraspBench** (Evaluation Framework): <https://github.com/JYChen18/DexGraspBench>
- **Dexonomy Dataset** (Taxonomy and Geometry Data): <https://huggingface.co/datasets/JiayiChenPKU/Dexonomy>

### A.2 System Requirements and Installation

To set up and run the GRIP-VR project, follow this step-by-step installation guide covering both the Python environment and the Unity project setup.

#### A.2.1 DexGraspBench and Dataset Setup

1. **Download DexGraspBench:** Clone or download the DexGraspBench repository from <https://github.com/JYChen18/DexGraspBench>. This repository contains the tools used to evaluate hand poses and create the dataset for this project.
2. **Download Dexonomy Dataset:** Download the Dexonomy dataset from <https://huggingface.co/datasets/JiayiChenPKU/Dexonomy>.
3. **Configure Assets:** Place the `DGN_5k` and `objaverse_5k` folders from the Dexonomy dataset inside the `assets` folder of DexGraspBench, as outlined in the DexGraspBench setup instructions.

### A.2.2 GRIP-VR and Server Setup

1. **Download GRIP-VR:** Clone the GRIP-VR repository inside the Dex-GraspBench directory from <https://github.com/mankarid/GRIP-VR>. The repository contains two main folders: **server** (Python scripts and FastAPI server) and **Unity** (the sample Unity project).
2. **Setup OpenShape:** Inside the **server** directory, download the OpenShape library from <https://huggingface.co/OpenShape/openshape-demo-support/tree/main> and follow their specific instructions to set it up.
3. **Create Embeddings Folder:** Within the **server/Llama** directory of GRIP-VR, create a new folder named **embeddings**. This folder will store all the object embeddings generated during runtime.

### A.2.3 Running the Inference Server

1. **Start the API Server:** Run the `api_server.py` script located in the **server** folder. This script initializes the FastAPI server, loads the model, and begins listening for incoming inference requests.

### A.2.4 Unity Project Setup and Execution

1. **Open the Unity Project:** Launch Unity and open the **Unity** folder from the GRIP-VR repository. This contains a sample scene pre-configured with a game object containing the necessary GRIP-VR scripts.
2. **Bake Grasps from AI Server:**
  - Ensure the Python API server is running in the background.
  - Select the pre-configured object in the Unity scene.
  - In the Inspector panel for the **Grasp Network Client** component, verify that the Server URL matches your active server’s IP and port.
  - Right-click the component or use the Context Menu to call the “**Bake all grasps from AI Server**” function. This will query the LLM and bake the Grasp Type Lists for the object.
3. **Run the VR Scene:**
  - Once the grasp lists are baked for all desired objects, you can safely close the Python server, as it has no effect during runtime and shutting it down saves local system resources.
  - Enter the Unity Scene using your VR headset.
  - Approach the configured object using hand tracking and attempt to grab it as you normally would. The UI inside the scene will dynamically display the grasp type prediction outputted by the GRIP-VR system.

## A.3 API Endpoints for the Python Server

The inference server exposes a FastAPI backend for real-time predictions. The primary endpoint utilized by the Unity XR client is `/predict_grasp`.

**Endpoint:** POST `/predict_grasp`

**Description:** Receives 3D object data alongside spatial constraints and scale, and returns a discrete list of valid Feix Taxonomy grasp types predicted by the fine-tuned LLaMA 3.2 model.

**Request Schema (GraspRequest):**

- **obj\_data** (string): The raw serialized `.obj` string representing the object's mesh geometry. Used to dynamically generate embeddings if not cached.
- **obj\_id** (string): A unique identifier for the object (e.g., "DGN\_5012"). Used for caching and retrieving embeddings.
- **object\_scale** (string): The physical scale of the object (e.g., "scale005").
- **scene\_constraints** (string): A textual representation of blocked approach vectors (e.g., "Blocked axes: -Y").

**Response Schema:**

- **top\_grasps** (list of strings): An array of semantic grasp labels derived from the Feix Taxonomy.
- **raw\_response** (string, optional): Included if the model's output cannot be successfully parsed into valid grasps.
- **error** (string, optional): Included if the prediction process encounters a critical error.

## A.4 Example JSON Payloads

### A.4.1 Example Request Payload

---

```
{
  "obj_data": "v -0.5 -0.5 0.5\nv 0.5 -0.5 0.5\n...",
  "obj_id": "DGN_5012",
  "object_scale": "scale005",
  "scene_constraints": "Blocked axes: -Y"
}
```

---

Listing A.1: Example JSON request sent from the Unity client to the FastAPI server.

#### A.4.2 Example Response Payload (Success)

---

```
{  
  "top_grasps": [  
    "17_Index_Finger_Extension",  
    "1_Large_Diameter",  
    "3_Medium_Wrap"  
  ]  
}
```

---

Listing A.2: Example JSON response returned by the server on a successful prediction.

#### A.4.3 Example Response Payload (Error)

---

```
{  
  "top_grasps": [],  
  "error": "OpenShape extraction failed"  
}
```

---

Listing A.3: Example JSON response returned by the server on failure.